

Deep Learning for NLP

Student name: *Margarita Orfanidi*

ID: *7115152400023*

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*

Semester: *Spring Semester 2025-2026*

Contents

1	Abstract	3
2	Analysis	3
2.1	Dataset	3
2.2	Exploratory Data Analysis	4
2.2.1	Samples	4
2.2.2	Descriptive Analysis	6
2.2.3	Textual Analysis	11
2.2.4	Token Analysis	14
3	Pre-processing	15
4	Models	16
4.1	Agents	17
4.1.1	Question Agent	17
4.1.2	Answer Agent	19
4.1.3	Gap and Evasion Agent	21
4.1.4	Decision Agent	23
5	Evaluation Metrics	25
6	Experiments	27
6.1	D3-Agentic Prompting Pipeline Example	27
6.2	Implementation Details	32
6.3	Output Parsing, Invalid Outputs and Fallbacks	33
7	Experimental Results	34
7.1	First set of Experiments	34
7.1.1	Baseline 1	34
7.1.2	Ablation Experiments	35
7.1.3	Prompt Formulation	38
7.2	Second Set of Experiments	46
7.2.1	Baseline 2	46
7.2.2	KNN Few-Shot Examples for the Decision Agent	47

7.2.3	DSPy Prompt Optimization	48
7.3	Overall Results	52
7.3.1	First Set	52
7.3.2	Second Set	53
8	Error Analysis	54
9	Best Model Selection	56
10	Comparison with the Previous Projects	57

1. Abstract

This assignment investigates response clarity classification in political interviews using the SemEval 2026 Task 6 CLARITY dataset. Each question–answer pair is classified as Clear Reply, Ambivalent, or Clear Non-Reply. The work focuses on D3-Agentic Prompting, a lightweight multi-agent framework implemented with an instruction-tuned language model.

All results, comparisons, error analysis, and discussion in this report are based on a full experimental run on the validation data, and every number reported here can be directly verified in the accompanying full-run notebook submitted on eClass (7115152400023-homework-4_Notebook_Full_run.ipynb), which retains the corresponding outputs. The same code is also available on Kaggle: the experiment cells are commented out to avoid re-running the expensive computations, but the pipeline is identical, and uncommenting and running these cells reproduces exactly the reported results. The final Kaggle submission, by contrast, was produced by running only the selected configuration on the official held-out test set at the end of the project; due to time and compute constraints, the complete set of experiments was not re-executed on the test data. Both the report and the full-run notebook were submitted on eClass.

2. Analysis

2.1. Dataset

The dataset consists of a collection of questions and answers from Presidents of the United States during interviews. The dataset is divided into two parts: a training set and a test set. The training set contains 3,448 entries with 20 attributes, while the test set contains 308 entries with the same attributes. The columns are described in the following table.

The dataset includes textual components such as the original interview question, the sub-questions, and the corresponding interview response. Moreover, it contains metadata, such as the interview title, date, and the name of the president. Finally, the dataset includes the proposed classification labels provided at the sub-question level, with each sub-question assigned classification labels (`clarity_label` and `evasion_label`) based on the proposed taxonomy.

It is important to note that the training set does not contain the individual labels assigned by each annotator, but only the final aggregated labels. Therefore, although the dataset generally contains non-null values, all values in these specific columns are null.

Column	Description
title	Interview title
date	Date of the interview
president	President being interviewed
url	Source URL of the interview
question_order	Order of the question in the interview
interview_question	Original interview question
interview_answer	Answer given by the president
gpt3.5_summary	Summary generated by GPT-3.5
gpt3.5_prediction	Prediction generated by GPT-3.5
question	Sub question
annotator_id	Identifier of the annotator
annotator1	Label from annotator 1
annotator2	Label from annotator 2
annotator3	Label from annotator 3
inaudible	Indicates if the segment contains inaudible parts
multiple_questions	Indicates if the question contains multiple parts
affirmative_questions	Indicates if the question is affirmative
index	Entry index
clarity_label	Label indicating response clarity
evasion_label	Label indicating response evasion

Table 1: Columns of the SemEval 2026 Task 6 dataset.

In contrast, in the test set, most metadata fields are null, while the columns necessary for evaluation are not null, such as the main text columns, the individual annotator labels (annotator1, annotator2, annotator3), and the corresponding labels.

2.2. Exploratory Data Analysis

Before developing the classification models, a study of the data and the nature of the problem must first be conducted. It is necessary to examine basic statistics, understand the distribution of the labels, and identify potential patterns and correlations in the data. Through this process, we can confirm the structure of the dataset, recognize the most important variables, and develop a clearer understanding of the problem we are trying to solve.

2.2.1. Samples. Random samples were displayed from the dataset in order to observe the structure and formatting of the textual data. This qualitative inspection helps identify potential patterns and understand how the text appears in practice, allowing us to draw conclusions about the preprocessing steps that may be required. For example, in the dataset, several spelling inconsistencies, missing spaces, and the presence of punctuation marks were observed, creating an initial intuition about the type of preprocessing that may be needed.

Sample 2900	
	Value
title	The President's News Conference With President Lee Myung-bak of South Korea at Camp David, Maryland
date	April 19, 2008
president	George W. Bush
url	https://www.presidency.ucsb.edu/documents/the-presidents-news-conference-with-president-lee-myung-bak-south-korea-camp-david
question_order	2
interview_question	Q. I have a question for President Lee. Korea and the United States have made many achievements through the summit meeting, especially North Korean nuclear issue and the strengthening of the alliance. As for North Korean nuclear issue, Mr. President Lee suggested setting up a permanent liaison office in both Seoul and Pyongyang. What are some of the follow-up effects, if you do have any follow-up actions? And do you have any thoughts of proposing a meeting with Chairman Kim at an earlier date?
interview_answer	The process is not something that we discussed between ourselves during the summit meeting. In fact, when I was staying in Washington, DC, I had an interview with one of the newspapers there, and it came up. Of course, it was not a sudden suggestion. I did have a meeting among my staff and related ministries, and I talked about this in detail before I came to the United States. We have a new administration in Korea, and we haven't yet begun dialogue with the ns. Inter-Korean dialogue, there is a need for us to have dialogue all the time. Up until now, we had dialogues whenever the need arose, and then it would stop. However, dialogue should be based on genuine cooperation and sincerity. And so with this in mind, I thought that it would be helpful to set up a permanent liaison office in both Seoul and Pyongyang. As for the summit meeting between myself and Chairman Kim, I will agree to it when the need is real. And I already said publicly that I am willing to meet with him—not just once, but many times—but if the meeting will yield substantial and real results. I believe only when that is possible, I am ready to meet with him and have sincere dialogue, because that will help to bring about peace and stability of the peninsula. So basically, I do hold that thought, but I'm not suggesting that—to have a meeting with Chairman Kim anytime soon. If the need arises, again, I'm ready to meet with him. Steven Lee [Steven Lee Myers, New York Times].
gpt3.5_summary	The question consists of 2 parts:\n1. Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.\n2. Thoughts on proposing a meeting with Chairman Kim at an earlier date.\n<-- Summary Answer -->\n\nThe response provides the following information regarding these points:\n1. Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue:\n- President Lee suggests setting up a permanent liaison office in both Seoul and Pyongyang to facilitate dialogue and cooperation between North Korea and South Korea.\n- The suggestion was made during an interview with a newspaper in Washington, DC, and was not discussed during the summit meeting.\n- The intention behind the suggestion is to establish genuine cooperation and sincerity in inter-Korean dialogue.\n2. Thoughts on proposing a meeting with Chairman Kim at an earlier date:\n- President Lee is open to the idea of a meeting with Chairman Kim, but only when the need is real and there is a possibility of substantial and real results.\n- He believes that sincere dialogue with Chairman Kim can contribute to the peace and stability of the Korean peninsula.\n- While he holds the thought of meeting with Chairman Kim, he is not suggesting a meeting anytime soon, but is willing to meet if the need arises.\n\nOverall, President Lee emphasizes the importance of genuine cooperation, sincerity, and substantial results in both the setup of a liaison office and the possibility of a meeting with Chairman Kim.
gpt3.5_prediction	Question 1: Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.\nVerdict: 1.1 Explicit - The information requested is explicitly stated (in the requested form).\n\nExplanation: The response explicitly provides information about President Lee's suggestion to set up a permanent liaison office in Seoul and Pyongyang. It explains that the suggestion was made to facilitate dialogue and cooperation between North and South Korea and highlights the intention behind the suggestion.\n\nQuestion 2: Thoughts on proposing a meeting with Chairman Kim at an earlier date.\nVerdict: 2.4 General - The information provided is too general/lacks the requested specificity.\n\nExplanation: The response does not specifically address the question of proposing a meeting with Chairman Kim at an earlier date. While President Lee expresses willingness to meet with Chairman Kim when the need is real and there can be substantial results, he does not provide a direct response to the request for thoughts on proposing an earlier meeting. The answer is more general in nature.
question	Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.
annotator_id	85
annotator1	None
annotator2	None
annotator3	None
inaudible	False
multiple_questions	False
affirmative_questions	True
index	2900
clarity_label	Ambivalent
evasion_label	Implicit

Figure 1: Sample

In addition, redundancy in the interview responses was observed. This occurs because some responses are similar despite referring to different questions, but mainly because the responses are labeled based on whether they address each sub-question. As a result, the same response may appear multiple times in the dataset, depending on the number of sub-questions associated with a particular interview question. This behavior is illustrated below by presenting the original question, the corresponding sub-questions, the interview response, and the assigned label. As shown, the same response is repeated across different sub-questions, while the assigned label may vary depending on the extent to which the response addresses each sub-question.

INTERVIEW QUESTION:

Q. And if I could, on the election, sir--can I--TikTok/Microsoft Corporation Q. President Trump, on the sale of TikTok, you're basically arguing that the U.S. Government is going to collect a cut from a--of a transaction including two companies, in which it doesn't hold a stake in. That's unprecedented. That's never happened in U.S. history before, and the administration has offered very little explanation about how that's going to work. Can you back your statement up and provide specifics about how that would work?

--- Example ---

Sub-question:

The interviewer's request for President Trump to back up his statement and provide specific details on how the government would collect this cut.

Answer:

Did you say, That's impressive? Did you actually use that term?...

Clarity label: Ambivalent

--- Example ---

Sub-question:

The previous comment made by the interviewer, expressing that President Trump is arguing for the U.S. Government to collect a cut from the sale of TikTok.

Answer:

Did you say, That's impressive? Did you actually use that term?...

Clarity label: Ambivalent

Figure 2: Sample

This redundancy can make class separation more difficult, since very similar or even identical answers may receive different labels depending on the specific sub-question being evaluated. As a result, relying only on the response text may not be sufficient, because the same linguistic content does not always imply the same degree of clarity.

2.2.2. Descriptive Analysis.

Clarity Labels Distribution The labels that were used for the classification task are the clarity labels, which include Clear Reply, Clear Non-Reply, and Ambivalent. The distribution of instances across these labels is shown in Figure 3. The majority of instances belong to the Ambivalent class, accounting for approximately 59.2% of the data (2039 samples). The Clear Reply category represents about 30.5% of the dataset (1051 samples), while Clear Non-Reply is the least frequent class with 10.3% (356 samples). This distribution indicates a noticeable class imbalance, with the Ambivalent class dominating the dataset, a factor that should be considered. Therefore, this imbalance should be considered during evaluation, and additional metrics such as macro-F1 are useful, since they give equal importance to all classes regardless of their frequency.

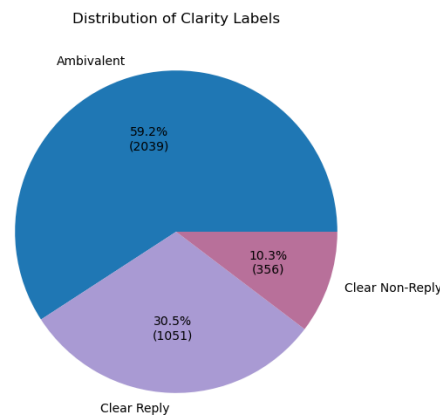


Figure 3: Distribution of clarity labels in the dataset

Evasion Labels Distribution The dataset also includes an `evasion_label`, which is a second, more detailed taxonomy that specifies the type of evasive strategy used in a response.

Figure 4 presents the relationship between clarity labels and evasion labels in the dataset. The evasion categories are Claims ignorance, Clarification, Declining to answer, Deflection, Dodging, Explicit, General, Implicit, and Partial/Half-answer. We can see that Deflection, Dodging, Implicit, General, and Partial/Half-answer correspond to Ambivalent responses, while Claims ignorance, Clarification, and Declining to answer correspond to Clear Non-Reply. The Explicit category corresponds to Clear Reply, meaning that the response is direct and clear.

evasion_label	Claims ignorance	Clarification	Declining to answer	Deflection	Dodging	Explicit	General	Implicit	Partial/half-answer
clarity_label									
Ambivalent	0	0	0	381	706	0	386	488	79
Clear Non-Reply	119	92	145	0	0	0	0	0	0
Clear Reply	0	0	0	0	0	1052	0	0	0

Figure 4: Relationship between clarity labels and evasion labels in the dataset.

Moreover, Figure 5 presents the distribution of the evasion labels in the dataset. The most frequent category is Explicit, accounting for 30.5% of the instances (1051 samples), which corresponds to clear and direct responses. Among the evasive strategies, Dodging appears most frequently (20.5%, 705 samples), followed by Implicit (14.2%, 487 samples), General (11.2%, 386 samples), and Deflection (11.0%, 380 samples). Less common categories include Declining to answer (4.2%), Claims ignorance (3.5%), Clarification (2.7%), and Partial/Half-answer (2.3%).

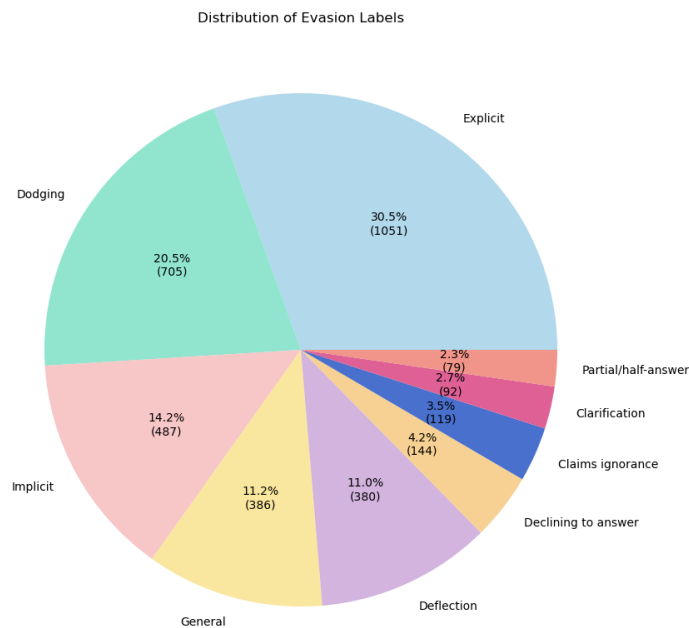
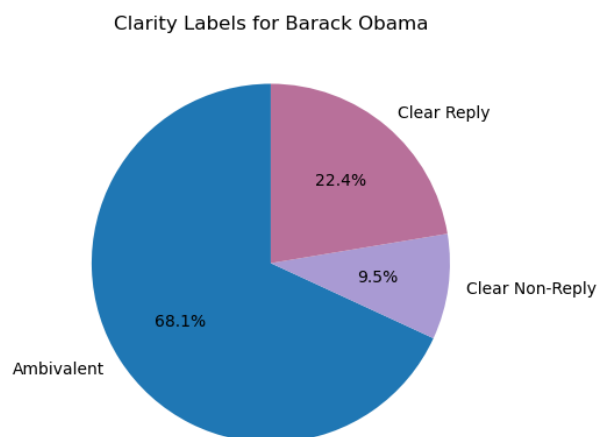
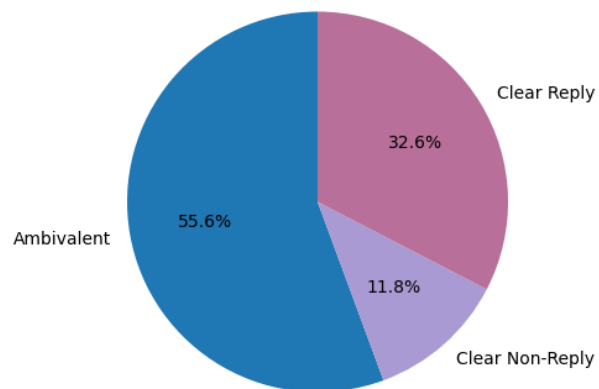


Figure 5: Distribution of evasion labels in the dataset.

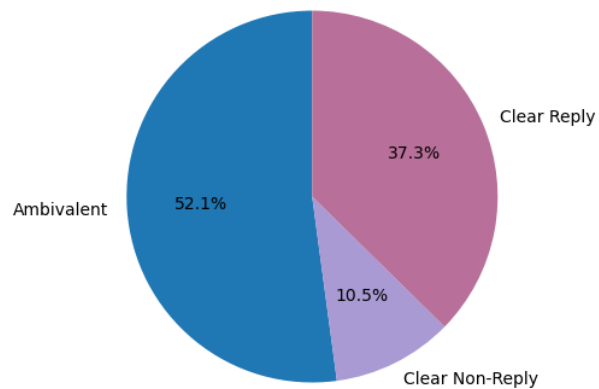
Clarity Label Distribution by President It is important to determine whether the nature of the responses in the dataset is speaker-dependent. An analysis of the distribution of clarity labels for each president helps reveal whether certain categories tend to appear more frequently depending on the president who provides the response, rather than representing a general characteristic of the dataset as a whole. As illustrated in the figures, Ambivalent responses constitute the majority for all presidents. However, some differences can still be observed. In particular, Barack Obama exhibits the highest proportion of Ambivalent responses (68.1%), indicating a larger share of indirect or partially evasive answers. In contrast, Donald J. Trump shows a higher proportion of Clear Reply responses (32.6%), suggesting that he provides more direct answers compared to the other presidents.



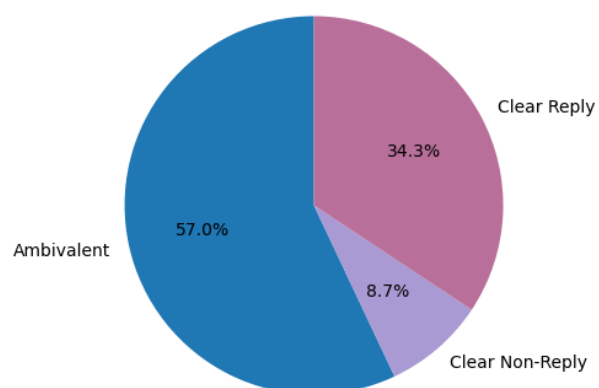
Clarity Labels for Donald J. Trump



Clarity Labels for Joseph R. Biden



Clarity Labels for George W. Bush



To further compare these distributions, the label frequencies across presidents are visualized using a grouped bar chart. As shown in the figure, although minor differences can be observed, no substantial variations appear in the overall response patterns

among the presidents. In all cases, Ambivalent responses remain the dominant category, followed by Clear Reply, while Clear Non-Reply appears less frequently. This suggests that the general response behavior is independent across the different presidents in the dataset.

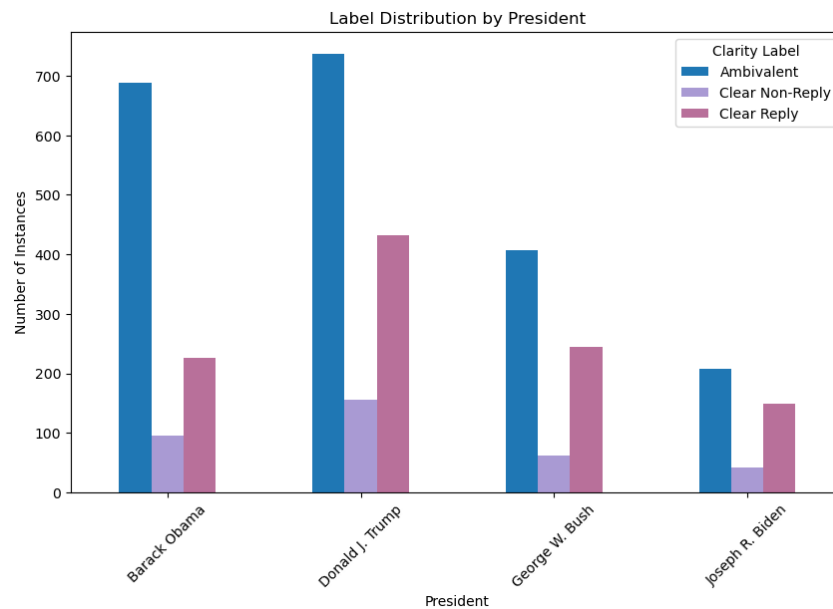


Figure 6: Label distribution per president

Clarity Labels Distribution by Affirmative questions Affirmative questions seem to be associated with slightly more direct responses. In these cases, Ambivalent responses become less frequent (from 60.5% to 54.5%), while Clear Reply (from 29.9% to 32.6%) and Clear Non-Reply responses (from 9.6% to 12.8%) become slightly more common. This suggests that affirmative questions may encourage clearer answers, although the effect remains limited.

clarity_label	Ambivalent	Clear Non-Reply	Clear Reply
affirmative_questions			
False	0.605007	0.096039	0.298954
True	0.545337	0.128238	0.326425

Figure 7: Label distribution per affirmative questions

Clarity Labels Distribution by Year Figure 8 shows a heatmap of interview questions by president and year. The distribution of questions matches the years of each presidency, with questions concentrated in the relevant time periods. Some differences can also be seen in the number of questions per year, since some years contain more instances than others.

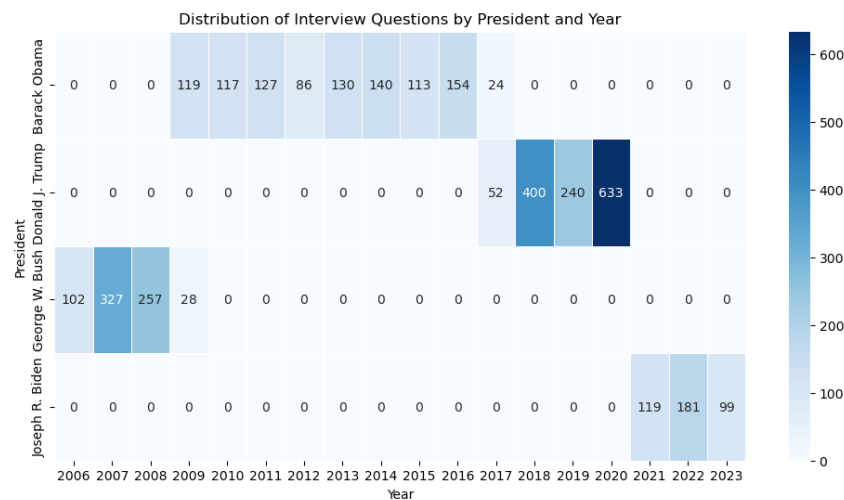


Figure 8: Distribution of interview answers per presidents and year

Figure 9 illustrates the distribution of clarity labels across different years. As observed, there is no significant temporal variation in the distribution, as the relative proportions of the labels remain stable over time.

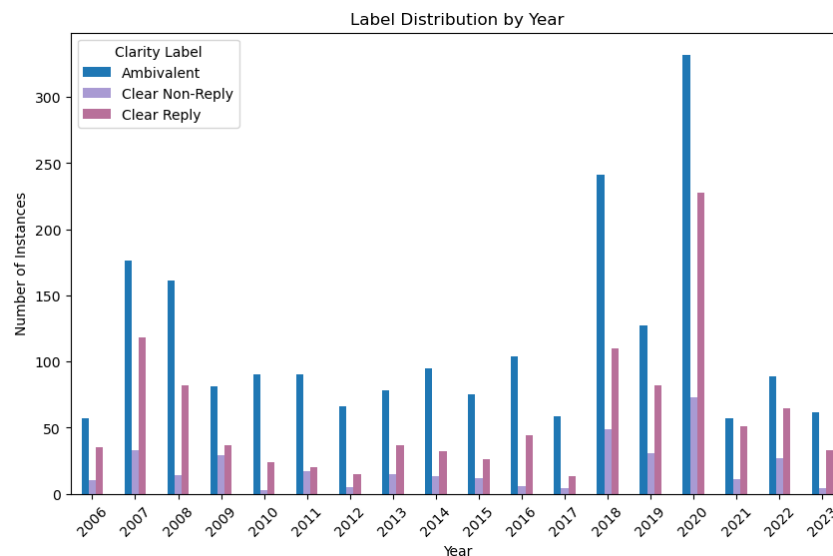


Figure 9: Distribution of clarity labels by year

2.2.3. Textual Analysis. The analysis then focused on the textual characteristics of the dataset. This step helps provide a better understanding of the language used in the interview answers, including aspects such as answer length, vocabulary, and recurring word patterns. It also helps identify whether simple textual properties differ across clarity labels or whether the classes share similar linguistic characteristics.

This is important for the current task because ambiguity is not always expressed through clear lexical cues. Similar vocabulary may appear across different classes, and the same answer may be interpreted differently depending on the question. Therefore, this analysis helps explain why the task is challenging and provides useful context for the design and evaluation of the prompting experiments.

Clarity Label Distribution by Word Count The analysis of answer length, measured in number of words, reveals notable differences across the clarity labels. The dataset contains 3,448 responses, with an average length of approximately 294 words and a high standard deviation (301.54), which indicates a variability in response length. The distribution is highly skewed, ranging from very short answers (1 word) to very long ones (up to 2,117 words). The median value is lower than the mean, suggesting the presence of long outlier responses that increase the average,.

When examining the classes separately, Ambivalent responses not only constitute the majority of the dataset but also tend to be longer on average (331.86 words). Clear Reply responses include 1,052 samples with a moderate average length of 272.04 words. In contrast, Clear Non-Reply responses form the smallest group and are considerably shorter, with an average length of 137.81 words.

Figure 10 illustrates these differences in answer length across the clarity labels. The boxplot shows that responses labeled as Ambivalent and Clear Reply tend to contain more words and exhibit greater variability in length compared to Clear Non-Reply. In contrast, Clear Non-Reply responses are generally shorter, with a lower median word count and a more concentrated distribution, although all classes contain several outliers.

Overall, these results suggest that answer length may be related to the clarity label. Shorter answers are more often associated with cases where the politician does not provide a substantial response to the question, while longer answers may include more context, indirect statements, or topic shifts, making it less clear whether the question has been fully answered. This observation suggests that response length may serve as a useful feature for distinguishing between clarity categories.

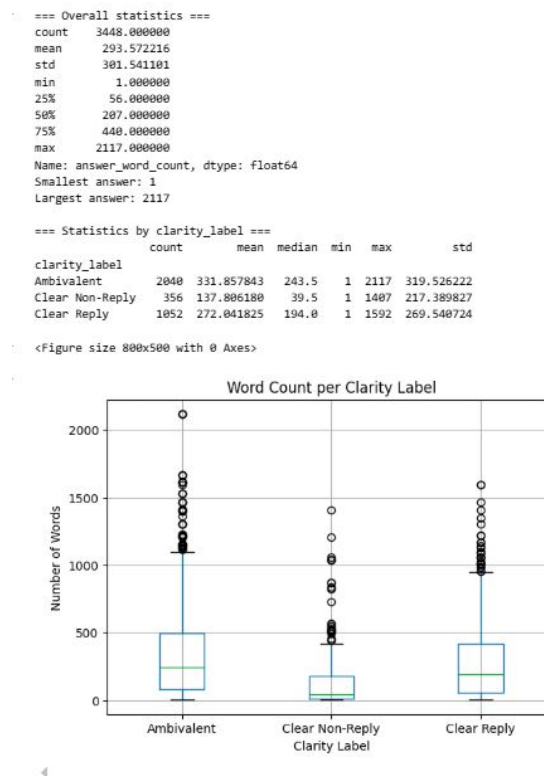


Figure 10: Words per label

Detection of text patterns in the dataset Although it is possible to manually inspect several examples from the dataset (e.g., by printing random entries), such inspection does not guarantee that all instances follow the same format. To systematically verify whether patterns are present in the dataset, a function named `detect_text_patterns` was implemented that scans the textual data and reports the occurrence of common types of noise. Specifically, the function searches for patterns such as HTML tags, email addresses, URLs, Twitter mentions, hashtags, digits, and newline characters using regular expressions. Additionally, it checks for the presence of emojis in the text. This automated inspection provides a quick overview of the dataset and helps determine whether specific cleaning operations are required before further processing. The results of this analysis indicated that the only notable pattern detected in the text column was the presence of numerical values.

```
detect_text_patterns(training_data["text"])
✓ 6.6s

Starting dataset inspection...
-----
Clean: No HTML Tags found.

Clean: No Emails found.

Clean: No URLs (http/www) found.

Clean: No Twitter Mentions (@) found.

Clean: No Hashtags (#) found.

Found: 1527 texts with Digits.
Clean: No Newlines (\n) found.

Clean: No Emojis found.

-----
Inspection completed!
```

Figure 11: Detection Patterns

Detection of spelling errors Moreover, a quality check was conducted to estimate the percentage of spelling errors present in the dataset. To do so, a function was implemented that scans the textual data and compares each word against an English dictionary using a spell-checking library. The function tokenizes the text into individual words, counts the total number of words, and identifies those that are not recognized by the dictionary as potential spelling errors. It then computes the percentage of misspelled words relative to the total number of words in the dataset. In addition, the function outputs a small set of example misspelled words to provide a qualitative indication of the types of spelling inconsistencies present in the data.

As shown in Figure 12, the analysis indicates that spelling errors constitute a very small portion of the dataset (approximately 1.96% of all words), suggesting that the textual data is generally clean and that spelling mistakes are unlikely to significantly affect the performance of the classification models. After removing stopwords, which usually carry limited semantic information for classification tasks, the percentage of misspelled words increased slightly to 2.94%. This suggests that spelling errors are somewhat more common in the more informative parts of the text, although their overall proportion remains relatively low.

```
check_spelling_errors(training_data["text"])  
✓ 4.4s  
Total words: 1125356  
Misspelled words: 22040  
Percentage of misspelled words: 1.96%  
Example misspelled words: ['thei', 'chineseno', 'aboutit', 've', 'donei']
```

Figure 12: Spelling errors

2.2.4. Token Analysis. This analysis was performed mainly to get an overview of how large the question–answer pairs become once tokenized with the Qwen tokenizer, and to check whether the token-length distribution differs between the training and test sets. This is a useful check, since a large mismatch in input length between the two sets could affect how reliably the behaviour observed on the training data transfers to the test set.

```
Total samples: 3448  
Mean length: 376.04  
Median length: 273.0  
Maximum length: 2465  
Minimum length: 5
```

Figure 13: Tokens

```
Total samples: 308  
Mean length: 412.91  
Median length: 379.5  
Maximum length: 1599  
Minimum length: 13
```

Figure 14: Tokens

The token statistics show that the test inputs are slightly longer on average than the training inputs. More specifically, the mean token length is 412.9 tokens for the test set compared to 376.0 tokens for the training set, while the median is also higher in the test set (379.5 vs. 273.0). This suggests a mild shift toward longer answers in the test set.

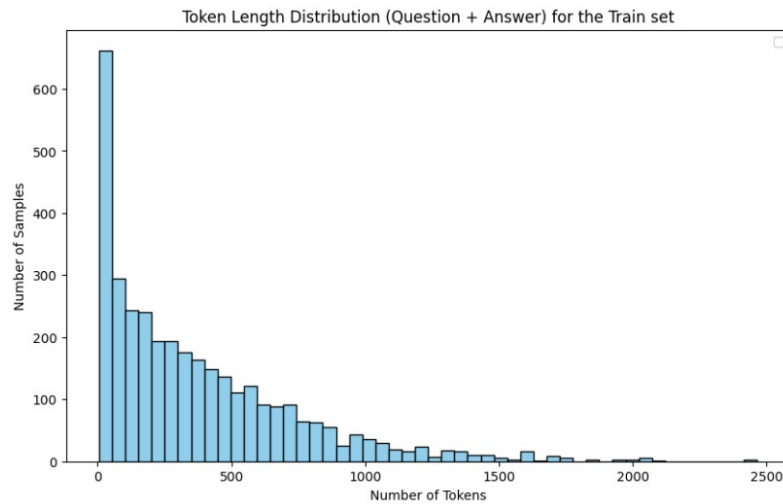


Figure 15: Tokens distribution

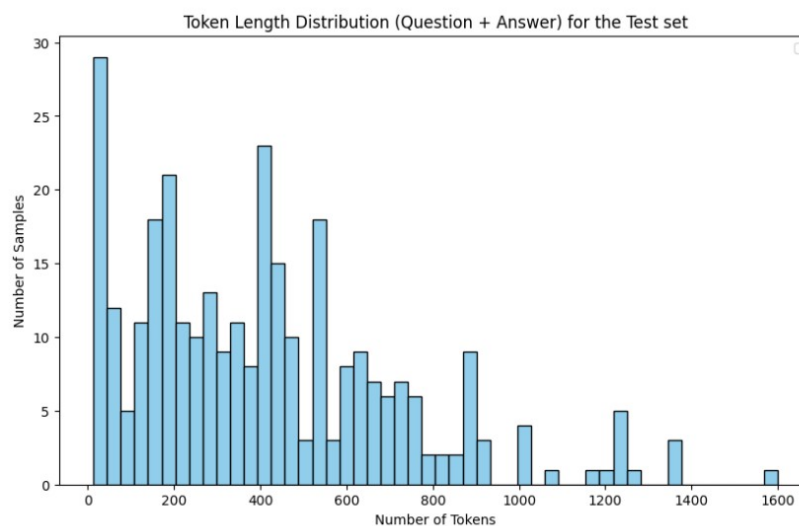


Figure 16: Tokens distribution

The histograms further support this observation, showing that both distributions are right-skewed. Most answers are relatively short, while a smaller number of responses are much longer and form a long tail. This pattern is particularly visible in the training set, where a large number of instances are concentrated in the lower token-length range. In contrast, the test set appears more spread out across medium and longer token lengths, which is consistent with its higher mean and median values.

3. Pre-processing

Based on the findings from the exploratory data analysis, a simple and targeted pre-processing pipeline was adopted. The pattern-detection step showed that the dataset was relatively clean, with no meaningful presence of HTML tags, URLs, hashtags, or

Twitter mentions, and the spelling analysis found that misspelled words accounted for less than 2% of the vocabulary. Since instruction-tuned large language models are generally robust to minor spelling variations, punctuation differences, and numerical values, no aggressive cleaning or spelling correction was applied, and the raw question and answer text was kept intact so that the models receive the same input a human reader would.

To reduce redundancy and make the experiments more efficient in terms of runtime, duplicate (question, interview_answer) pairs were removed from the training set. After this step, the training set was reduced from 3,448 to 3,390 rows.

```
Before drop_duplicates: (3448, 21)
After drop_duplicates: (3390, 21)
```

Figure 17: Number of training rows after duplicate removal

After duplicate removal, the dataset was split into two disjoint subsets using a stratified hold-out strategy. The larger subset, `train_part`, was used as the pool of examples for the prompting experiments. The smaller subset, `valid_df`, was used as the experimental evaluation set for the reported validation metrics.

4. Models

LLMs Large Language Models (LLMs) are artificial-intelligence systems trained on very large amounts of textual data with the objective of predicting the next token in a sequence. Most LLMs are based on neural networks, and a key element behind their strong performance is the attention mechanism, which allows the model to focus on different parts of the input when producing each part of the output.

Prompting Prompting is the technique of providing instructions in natural language to an LLM in order to guide its output. Its main advantage is that the task is specified entirely through the prompt, so the model can be steered toward a particular problem without any additional training or fine-tuning. As a consequence, the wording, structure, and content of the prompt can strongly affect the quality and consistency of the response. Two main prompting methods are : zero-shot prompting and few-shot prompting. In zero-shot prompting, the model is asked to solve the task without being given any examples. In few-shot prompting, a small number of labelled examples are included in the prompt to demonstrate the expected behaviour. These methods aim to make the model's output more controlled and structured, especially for complex classification tasks.

Agentic prompting Agentic prompting extends standard prompting by organizing the use of an LLM into a more structured decision-making process. Instead of asking the model to solve the whole task with a single prompt, the task is decomposed into smaller subtasks, each guided by a specialized instruction. In this setting, the LLM acts as the reasoning component of an agent, while the prompt defines the agent's role, objective, input format, and expected output. This approach can improve interpretability,

because intermediate outputs show how the system analyzes the input before reaching a final decision. It is also useful for complex classification tasks, where different aspects of the input need to be examined separately before assigning a label.

Models The evaluation was conducted on one instruction-tuned LLM from the Qwen3.5 family

Table 2: Model

Model Name	Total Parameters	Context Length	Data Type	Vocab Size
Qwen/Qwen3.5-0.8B	752,393,024 (0.75B)	262,144 tokens	bfloat16	248,320

4.1. Agents

To classify the responses of presidents it was composed a D3-Agentic Prompting which is a multi-agent prompting framework. This framework decomposes the task of the response clarity classification into a sequence of steps.

Rather than using a single prompt to analyze the question–answer pair and directly predict the final label, the task is divided among four agents, each responsible for a specific part of the decision process.

This design allows the model to examine the input more systematically and produces intermediate outputs that make the classification process more interpretable.

The pipeline consists of four agents:

- **Question Intent Agent:** identifies what the question is asking and what information would be required for a clear answer.
- **Answer Content Agent:** summarizes what the answer actually states, focusing on explicit claims and relevant information.
- **Gap and Evasion Agent:** compares the expected answer content with the actual answer and detects missing information, ambiguity, partial answers, or evasive patterns.
- **Decision Agent:** uses the outputs of the previous agents to assign one of the three final labels: Clear Reply, Ambivalent, or Clear Non-Reply.

The following describes the baseline agent pipeline in detail, presenting the exact prompts used for each agent. Note that, across the experiments in which different configurations were tested, minor variations of these prompts may occur.

4.1.1. Question Agent . The first agent in the D3-Agentic Prompting pipeline is the Question Intent Agent. This agent receives as input the target question from the clarity classification dataset. Since the target question may be only one sub-question extracted from a longer interview question, the agent is instructed to analyze only the given target question and not require the answer to address any other surrounding questions unless they are necessary for understanding the target question itself. Its role is to

identify exactly what kind of information would make an answer clear for that specific question.

More specifically, the Question Intent Agent determines whether the question asks for a yes/no answer, what happened, why or how something happened, a specific action, plan, reason, explanation, stance, evaluation, or information about a named person, event, country, policy, organization, or date. This step helps define the expected content of a clear answer before the actual interview response is analyzed by the following agents.

To respect the dataset structure, the agent analyzes only the `TARGET_QUESTION`, which may be a single sub-question extracted from a longer interview question, without requiring the answer to address unrelated questions. The agent returns its analysis in a structured JSON format, including the following fields:

- **question_type:** the category of the question (e.g., *yes_no*, *what*, *why*, *how*, *when*, *where*, *who*, *stance*, *explanation*, *action_plan*, *evaluation*, or *other*).
- **question_intent:** a one-sentence description of what the target question asks.
- **must_address:** a list of the specific requirements that must be addressed for the answer to be considered relevant.
- **clear_answer_criteria:** a one-sentence description of what a clear answer must contain.
- **minimal_sufficient_answer:** a short description of the minimum information needed for a response to be classified as a *Clear Reply*.

```
"""
You are the Question Intent Agent for a response-clarity classification task.

Important dataset rule:
The TARGET_QUESTION may be only one sub-question extracted from a longer
interview question.
Analyze ONLY the TARGET_QUESTION. Do not require the answer to address other
questions unless they are necessary for this target question.

Your task:
Identify exactly what information would make an answer clear for the
TARGET_QUESTION.

Think in terms of answer requirements:
- Is the question asking for yes/no?
- Is it asking what happened?
- Is it asking why/how something happened?
- Is it asking for a specific action, plan, reason, explanation, stance, or
evaluation?
- Is it asking about a named person, event, country, policy, organization, or
date?

Return exactly one JSON object and nothing else.
```

```

JSON schema:
{
  "question_type": "yes_no|what|why|how|when|where|who|stance|explanation|
    action_plan|evaluation|other",
  "question_intent": "one sentence describing what the target question asks",
  "must_address": [
    "specific requirement 1",
    "specific requirement 2"
  ],
  "clear_answer_criteria": "one sentence describing what a clear answer must
    contain",
  "minimal_sufficient_answer": "short description of the minimum information
    needed for Clear Reply"
}
"""

```

4.1.2. Answer Agent. The second agent in the D3-Agentic Prompting pipeline is the Answer Content Agent. This agent receives the target question and the corresponding interview answer as input. Since the interview answer may have been given in response to a longer interview question, the agent is instructed to evaluate the answer only with respect to the given target question. Its role is not to assign the final clarity label, but to extract what the answer actually says that is relevant to the target question.

More specifically, the Answer Content Agent identifies whether the answer contains a direct response, such as a yes/no statement, a reason, an explanation, an action, or a commitment. It also extracts concrete relevant information, separates vague or general statements from specific content, and detects off-target parts of the answer, such as topic shifts, refusals, political rhetoric, or responses to a different sub-question. The agent is explicitly instructed to describe only what the answer actually contains, without inferring information that is not present and without judging whether the answer is good, evasive, or complete, since that is handled by a later agent. Its output is returned in a structured JSON format, including the following fields:

- **relevant_answer_summary:** a one-sentence summary of only the content that is relevant to the target question.
- **direct_answer_present:** a boolean indicating whether the answer contains a direct response to the target question.
- **direct_answer_text:** the exact short phrase that directly answers the question if present, otherwise an empty string.
- **concrete_information:** a list of concrete, on-topic points relevant to the question, such as facts, actions, reasons, or examples.
- **vague_or_general_information:** a list of vague or general statements that are related but lack specific content.
- **off_target_content:** a list of parts that answer a different question or shift the topic.

- **refusal_or_non_answer_signals**: a list of signals indicating refusal, declining, or non-answer.

```
"""
You are the Answer Content Agent for a response-clarity classification task.

Important dataset rule:
The answer may respond to a longer interview question, but you must evaluate
    what it says with respect to the TARGET_QUESTION only.

Your task:
Extract what the answer actually says that is relevant to the TARGET_QUESTION.

Do not decide the final label.
Do not reward political rhetoric unless it answers the target question.
Do not infer facts that are not present in the answer.
An implicit answer is allowed only if the answer gives enough information to
    resolve the target question.

Look for:
- direct answer phrases: yes, no, I did, I did not, we will, we will not,
    because...
- concrete facts, actions, reasons, explanations, examples, commitments
- vague but related statements
- topic shifts
- refusals or declines to answer
- answers to a different sub-question

Return exactly one JSON object and nothing else.

JSON schema:
{
  "relevant_answer_summary": "one sentence summarizing only content relevant
    to the target question",
  "direct_answer_present": true,
  "direct_answer_text": "exact short phrase if present, otherwise empty string",
  "concrete_information": [
    "concrete relevant point 1",
    "concrete relevant point 2"
  ],
  "vague_or_general_information": [
    "vague/general relevant point 1"
  ],
  "off_target_content": [
    "content that answers a different question or changes topic"
  ],
  "refusal_or_non_answer_signals": [
    "signal 1"
  ]
}
```

```
"""
```

4.1.3. Gap and Evasion Agent. The third agent in the D3-Agentic Prompting pipeline is the Gap and Evasion Agent. This agent receives as input the target question, the interview answer, the output of the Question Intent Agent, and the output of the Answer Content Agent. Its role is to compare what the target question requires with what the answer actually provides. In other words, it examines whether the relevant content extracted from the answer is sufficient to satisfy the intent and requirements identified for the question.

More specifically, the Gap and Evasion Agent identifies the degree of coverage between the expected answer and the provided answer. It distinguishes between cases where the answer is complete, mostly complete, partial, minimal, or not relevant at all, and characterizes the seriousness of the gap as none, minor, partial, or major. This is important because an answer may contain some relevant information but still fail to fully address the target question, for example by answering only part of the question, giving a vague or generic response, redirecting to another topic, refusing to answer, claiming lack of knowledge, or providing information that is related but not sufficient.

The agent also detects possible evasion patterns using categories such as explicit answer, implicit answer, partial answer, general answer, dodging, deflection, declining to answer, claims of ignorance, clarification only, or irrelevant response. However, it does not assign the final clarity label. Instead, it produces a structured JSON output including the following fields:

- **coverage:** the degree of coverage between expected and provided answer (*complete, mostly_complete, partial, minimal, or none*).
- **gap_level:** the seriousness of the gap (*none, minor, partial, or major*).
- **best_evasion_category:** the most fitting evasion pattern (e.g., *explicit_answer, implicit_answer, partial_half_answer, general_answer, dodging, deflection, declining_to_answer, claims_ignorance, clarification_only, or irrelevant*).
- **missing_requirements:** a list of requirements from the question that the answer fails to address.
- **evidence_for_answering:** short evidence from the answer that it addresses the question.
- **evidence_for_evasion_or_gap:** short evidence from the answer indicating a gap or evasion.
- **gap_explanation:** one concise sentence explaining the gap.

```
"""
```

```
You are the Gap and Evasion Agent for response-clarity classification.
```

```
You receive:
```

- ```
1. the TARGET_QUESTION,
2. the answer,
3. the Question Intent Agent output,
```

#### 4. the Answer Content Agent output.

Your task:

Compare what the target question requires with what the answer provides.

Important distinction:

- Missing small detail but mostly answering -> minor gap.
- Some relevant information but vague, incomplete, generic, or indirect -> partial gap.
- Mostly answering another topic, refusing, or avoiding the target question -> major gap.

Use these evasion categories when useful:

- explicit\_answer: directly answers the target question.
- implicit\_answer: not in requested form, but enough information is provided.
- partial\_half\_answer: answers only part of what was asked.
- general\_answer: gives broad values/principles but lacks the requested specifics.
- dodging: avoids the precise target question while saying related things.
- deflection: redirects to another issue/person/topic.
- declining\_to\_answer: refuses, says cannot comment, or avoids due to confidentiality.
- claims\_ignorance: says they do not know or cannot assess.
- clarification\_only: asks for clarification or corrects framing without answering.
- irrelevant: no meaningful relation to the target question.

Do not assign the final top-level label.

Return exactly one JSON object and nothing else.

JSON schema:

```
{
 "coverage": "complete|mostly_complete|partial|minimal|none",
 "gap_level": "none|minor|partial|major",
 "best_evasion_category": "explicit_answer|implicit_answer|
 partial_half_answer|general_answer|dodging|deflection|declining_to_answer|
 claims_ignorance|clarification_only|irrelevant",
 "missing_requirements": [
 "missing requirement 1"
],
 "evidence_for_answering": [
 "short evidence from the answer"
],
 "evidence_for_evasion_or_gap": [
 "short evidence from the answer"
],
 "gap_explanation": "one concise sentence"
}
```

**4.1.4. Decision Agent.** The final agent in the D3-Agentic Prompting pipeline is the Decision Agent. This agent receives the target question, the interview answer, and the intermediate outputs produced by the previous agents. Its role is to assign the final response-clarity label by deciding how clearly the answer responds to the target question. The agent follows the dataset rule that only the target question should be judged, not the whole interview question. Therefore, if the answer responds to another sub-question but does not address the target question, it should not be classified as a Clear Reply.

To make the decision consistent, the prompt encodes explicit rules that map the coverage and gap analysis of the previous agent to a label: complete or mostly complete coverage yields *Clear Reply*, partial or minimal coverage with relevant information yields *Ambivalent*, and no coverage or clear evasion yields *Clear Non-Reply*.

The prompt also includes few-shot calibration examples to guide the model in borderline cases, such as indirect but sufficient answers, vague political statements, refusals, and answers that address the wrong person or sub-question. The agent returns its final decision in a structured JSON format, including the following fields:

- **label:** the predicted clarity label (*Clear Reply*, *Ambivalent*, or *Clear Non-Reply*).
- **confidence:** a confidence score for the assigned label.
- **main\_reason:** one concise sentence explaining the main reason for the decision.
- **used\_rule:** a short description of the rule that determined the label.

```
"""
You are the Decision Agent for CLARITY response classification.

Assign exactly one final label for how clearly the ANSWER responds to the
TARGET_QUESTION.

Valid labels:
1. Clear Reply
2. Ambivalent
3. Clear Non-Reply

Critical dataset rule:
Judge only the TARGET_QUESTION, not the whole interview question. If the
answer addresses a different sub-question but not the target question,
this is not a Clear Reply.

Definitions:

Clear Reply:
- The answer directly or implicitly provides the information requested by the
target question.
- A yes/no question can be Clear Reply if the answer clearly implies yes or no
.
- The answer does not need to be long.
- The answer may be politically framed, but it must still resolve the target
question.
```



**Ambivalent:**

- The answer contains some relevant information but is incomplete, vague, overly general, or only partially responsive.
- The answer gives values, background, context, or related claims but does not fully satisfy the target question.
- The answer may answer part of a multi-part question but misses the current target question.
- Use Ambivalent for borderline cases where there is meaningful relevance but insufficient clarity.

**Clear Non-Reply:**

- The answer does not meaningfully answer the target question.
- It refuses to answer, says it cannot comment, claims ignorance, redirects to another topic, or only answers a different question.
- Use Clear Non-Reply when the response gives little or no usable information for the target question.

**Decision rules:**

- If coverage is complete or mostly\_complete and the answer resolves the target question -> Clear Reply.
- If coverage is partial/minimal but there is relevant information -> Ambivalent.
- If coverage is none, or the response is mainly refusal/deflection/irrelevant -> Clear Non-Reply.
- Do not classify as Clear Non-Reply only because the answer is indirect. Indirect but sufficient answers are Clear Reply.
- Do not classify as Clear Reply only because the answer is fluent or politically detailed. It must answer the target question.
- Prefer Ambivalent over Clear Non-Reply when the answer has relevant but insufficient information.
- Prefer Ambivalent over Clear Reply when the answer is related but lacks the requested specific stance/action/reason.

**Few-shot calibration examples:****Example A:**

Target question: "Did you support the bill?"

Answer content: "Yes, I voted for it and I still support it."

Gap analysis: coverage complete; explicit answer.

Final label: Clear Reply.

**Example B:**

Target question: "What specific steps will you take to reduce prices?"

Answer content: "Families are struggling, this is important, and we are working every day."

Gap analysis: relevant but no specific steps.

Final label: Ambivalent.

**Example C:**

Target question: "Did you meet with the ambassador?"



Answer content: "I am not going to discuss private diplomatic conversations."  
 Gap analysis: declining to answer; no requested information.  
 Final label: Clear Non-Reply.

Example D:

Target question: "Why did unemployment increase?"  
 Answer content: "The economy has faced global pressure, energy shocks, and reduced demand."  
 Gap analysis: gives reasons even without saying 'because'.  
 Final label: Clear Reply.

Example E:

Target question: "Do you think President X is sincere?"  
 Answer content: "I am sincere about improving the relationship."  
 Gap analysis: answers the speaker's sincerity, not President X's sincerity.  
 Final label: Ambivalent.

Return exactly one JSON object and nothing else.

JSON schema:

```
{
 "label": "Clear Reply|Ambivalent|Clear Non-Reply",
 "confidence": 0.0,
 "main_reason": "one concise sentence",
 "used_rule": "short description of the rule that determined the label"
}
```

## 5. Evaluation Metrics

To evaluate the agentic system on the clarity classification task, the predicted labels produced by the Decision Agent were compared against the gold clarity labels. The evaluation relies on standard classification metrics. A detailed classification report is produced for the three valid classes, Clear Reply, Ambivalent, and Clear Non-Reply, reporting precision, recall, and F1-score per class together with macro and weighted averages. In addition, the main summary metrics are reported separately: accuracy, macro-averaged precision, recall, and F1, and weighted-averaged precision, recall, and F1. All metrics are computed with `zero_division=0`, so that a class with no predicted samples contributes a zero score instead of raising an error.

**Mapping free-form outputs to labels.** Because the underlying model is generative, the Decision Agent does not directly emit one of the predefined classes. Its raw output is first parsed as a JSON object, and the value of the label field (or, if parsing fails, the raw text) is passed through a normalisation step. This step lowercases the text and matches it against the three target labels and a small set of synonyms (for example, *ambiguous*, *partial*, or *unclear* are mapped to Ambivalent, and variants such as *non reply* are mapped to Clear Non-Reply). If no label can be recognised, the prediction falls back to Ambivalent, which is the majority class; the same fallback is applied whenever the pipeline raises an error for a given sample. As a consequence, every sample always

receives one of the three valid labels, and no separate Invalid category arises in the agentic setting.

**Effect of the fallback on the metrics.** Since unparsable or off-format generations are mapped to the majority class Ambivalent rather than to a dedicated Invalid category, all metrics are computed over the three valid labels. This makes the evaluation slightly more lenient than a strict format-checking scheme: a malformed output counts as correct whenever the gold label is Ambivalent, and as an ordinary wrong-label error otherwise. Formatting failures are thus absorbed into the majority class, where they primarily inflate the number of predicted Ambivalent samples. Their main effect is therefore to lower the precision of Ambivalent and the recall of Clear Reply and Clear Non-Reply.

**Inspection of invalid generations.** Even though normalisation guarantees a valid label for every sample, the raw outputs of the Decision Agent are inspected separately to flag cases where the model deviates from the expected response format or produces text that is hard to parse. These cases are not scored as a distinct class, since the fallback already handles them; rather, the inspection serves as a diagnostic to distinguish genuine wrong-label decisions from purely formatting-related errors. A handful of invalid outputs are additionally examined by hand, alongside their rationales, to trace the source of specific mistakes.

**Choice of headline metric.** Among the F1-based metrics, macro-F1 is treated as the most informative for this task because the dataset is imbalanced: Ambivalent is the majority class while Clear Non-Reply is the minority class. By giving equal weight to all classes, macro-F1 reflects performance across the full label set rather than rewarding strong performance on the dominant class. Weighted-F1 is also reported, since it is the official metric of the associated Kaggle competition and the score on which the final system is ranked. However, because it weights each class by its frequency, a model can still obtain a relatively high weighted-F1 even when it performs poorly on the minority class.

**Confusion matrix.** The confusion matrix is used to inspect the types of mistakes made by the system. It is computed over the three valid classes, so both axes contain Clear Reply, Ambivalent, and Clear Non-Reply. This makes it possible to observe which classes are most often confused with one another, for example whether Clear Non-Reply answers tend to be misclassified as Ambivalent.

**Error analysis.** A finer-grained analysis of the mistakes is performed on the validation predictions. All misclassified samples are first grouped by their error pattern, defined as the pair of gold and predicted label, and the most frequent (true  $\rightarrow$  predicted) confusion patterns are reported. Because the normalisation step maps every model output to one of the three valid labels, invalid or off-format generations are not scored as a separate class; in the scored predictions, they appear as ordinary wrong-label errors whenever the fallback label differs from the gold label. Beyond the aggregate counts, the raw Decision Agent outputs and rationales are inspected qualitatively, which makes it possible to see whether a mistake stems from the final decision prompt,

from an off-format generation, or from misleading intermediate outputs produced by the earlier agents.

**Error rate by answer length.** Performance is also analysed with respect to answer length. The interview answers are split into four quantile-based buckets of approximately equal size, using the word count as the length measure, and the error rate is computed for each bucket; the same breakdown is additionally produced for question length. This analysis checks whether classification errors become more frequent for shorter or longer inputs. Very short answers may provide too little evidence for a reliable decision, whereas longer political answers may introduce more context, topic shifts, vague statements, or off-target content. Comparing error rates across length buckets therefore indicates whether length is associated with the system's mistakes.

## 6. Experiments

### 6.1. D3-Agentic Prompting Pipeline Example

The full pipeline as it mentioned consists of four agents:

1. **Question Intent Agent**
2. **Answer Content Agent**
3. **Gap and Evasion Agent**
4. **Decision Agent**

Each agent is implemented as a separate prompt to the same underlying Qwen3.5-0.8B model. The model was loaded using the Hugging Face transformers library with AutoTokenizer and AutoModelForCausalLM. The tokenizer was configured with left padding, which is suitable for decoder-only language models during batched generation. If the tokenizer did not define a padding token, the end-of-sequence token was reused as the padding token.

The output of each upstream agent is passed as additional context to the following agents. In this way, the final decision is not based only on the raw question and answer, but also on an accumulated analysis of the target question, the answer content, and the gap between them.

To make the pipeline easier to understand, we use one example from the dataset. The question asks for a clear yes/no answer: whether the speaker guarantees that the affected people will continue receiving federal financial help to pay for their insurance.

Question:

Are you going to ensure that those people will still get help from the Federal Government to pay that insurance?

Answer:

Well, that's what we're looking to do, Joe [Jonathan Karl, ABC News]. We want to get it down so that people can have affordable health care. Look, you look at some States 116 percent up. In Alaska, over 200 percent up. In other States, 50 percent, 70 percent up, and those are some of the States

that are doing better. Obamacare is a wreck, it's a mess, it's destroying lives. We want to get it in those States the States that I did so well in but also in States that I didn't win. I want to get health care that's much more affordable and much better health care, and that's what we're doing. Yes.

Label:

Ambivalent

The answer ends with “Yes”, but most of the response does not give a clear guarantee. Instead, the speaker talks generally about making health care more affordable, criticizes the current system, and mentions premium increases. For this reason, the gold label is Ambivalent, because the answer is somewhat related but does not clearly confirm or reject the guarantee.

The following analysis shows how each of the four agents processes this example and explains why the pipeline ultimately assigns a label that differs from the gold label.

**Question Intent Agent Output** The Question Intent Agent is the first step of the pipeline. Its job is to understand what the target question is asking and what a clear answer should include.

In this example, the Question Intent Agent partially identifies the main requirement of the target question, since the clear answer criteria and the minimal sufficient answer both refer to whether federal financial help will continue. However, the output is not fully correct. The agent does not properly fill the `must_address` field and instead copies placeholder requirements from the prompt. It also fails to select a single question type, copying part of the list of possible types instead. Therefore, the first stage already introduces some noise into the pipeline, even though it captures the general direction of the question.

```
=== QUESTION INTENT PARSED JSON ===
{
 "question_type": "yes_no|what",
 "question_intent": "Are you going to ensure that those people will still get
 help from the Federal Government to pay that insurance?",
 "must_address": [
 "specific requirement 1",
 "specific requirement 2"
],
 "clear_answer_criteria": "The answer must clearly state whether the speaker
 is going to ensure that the people will still receive help from the
 Federal Government to pay the insurance.",
 "minimal_sufficient_answer": "Yes, I am going to ensure that those people
 will still get help from the Federal Government to pay that insurance."
}
```

**Answer Content Agent Output** The Answer Content Agent receives the question-answer pair and extracts what the answer appears to say. In this example, the agent correctly identifies that the answer contains some relevant material about health care, insurance premiums, financial help, and the current health care system. It also correctly detects that the speaker uses vague and general language, since the answer talks

about “help” and “health care” without clearly explaining the exact mechanism of the financial assistance.

```

=== ANSWER CONTENT PARSED JSON ===
{
 "relevant_answer_summary": "The speaker confirms that the goal is to ensure
 people will still get help from the Federal Government to pay their
 insurance.",
 "direct_answer_present": true,
 "direct_answer_text": "Yes.",
 "concrete_information": [
 "The speaker explicitly states the goal is to ensure people will still get
 help from the Federal Government to pay their insurance.",
 "The speaker mentions specific examples of states that are doing better,
 such as Alaska (over 200 percent up) and other states (50-70 percent up).",
],
 "vague_or_general_information": [
 "The speaker uses general terms like 'States' and 'people' to describe the
 scope of the goal."
],
 "off_target_content": [],
 "refusal_or_non_answer_signals": []
}

```

However, this stage also introduces some errors. The agent identifies the final “Yes.” as a direct answer and therefore sets `direct_answer_present` to true. Although this is understandable, because the word “Yes” does appear in the answer, it is not fully reliable in context. The response before this final word is long, indirect, and mostly focused on broader health-care issues rather than on the specific guarantee requested in the question.

The agent also gives a somewhat overly positive summary of the answer. For example, it interprets the references to premium increases, such as “116 percent up” and “over 200 percent up”, as evidence that states are improving their ability to afford health care. In the original answer, however, these numbers are used mainly to criticize the existing health-care system. This makes the extracted content appear more supportive and responsive than it actually is.

Another limitation is that the fields `off_target_content` and `refusal_or_non_answer_signals` are left empty. This is important because several parts of the response, including the criticism of Obamacare, the discussion of premium increases, and the general statements about affordability, do not directly answer whether federal financial help will continue.

Therefore, the Answer Content Agent does capture some relevant elements of the response, but it presents the answer as clearer and more direct than it really is. This imperfect interpretation is then passed to the next stages of the pipeline.

**Gap and Evasion Agent Output** The Gap and Evasion Agent receives the target question, the original answer, and the outputs of the previous agents. Its role is to compare what the question asks with what the answer provides, and to identify whether there is a gap or an evasive pattern.

In this example, the agent is influenced by the imperfect outputs of the previous

stages. First, the Question Intent Agent produces an imperfect requirement description, which adds some noise to the comparison. More importantly, the Answer Content Agent gives an overly positive interpretation of the answer by treating the final “Yes” as a clear direct answer and by not marking much of the response as off-target.

As a result, the Gap and Evasion Agent labels the coverage as `mostly_complete`, the gap level as `minor`, and the evasion category as `explicit_answer`. This means that it treats the response as mostly answering the target question, with only a small missing detail.

However, this interpretation is too optimistic. The answer does not clearly guarantee that federal financial assistance will continue. Instead, it mostly discusses general health-care affordability, premium increases, and criticism of the current system. Because the previous agents made the answer appear more direct and complete than it really was, the Gap and Evasion Agent underestimates the gap between the question and the answer.

```
=== GAP EVASION PARSED JSON ===
{
 "coverage": "mostly_complete",
 "gap_level": "minor",
 "best_evasion_category": "explicit_answer",
 "missing_requirements": [
 "Specific requirement 2"
],
 "evidence_for_answering": [
 "The answer explicitly states: 'Yes, I am going to ensure that those people will still get help from the Federal Government to pay that insurance.'"
],
 "evidence_for_evasion_or_gap": [
 "The answer provides a direct affirmative response to the core question, but the specific requirement 2 (the exact mechanism or method of payment) is not explicitly detailed in the text."
],
 "gap_explanation": "The answer provides a direct affirmative response to the core question, but the specific requirement 2 (the exact mechanism or method of payment) is not explicitly detailed in the text."
}
```

During the inspection of the raw Gap and Evasion Agent outputs, I noticed that the model did not always return valid JSON. In some cases, the generated content was useful, but the JSON structure was incomplete. For example, the model sometimes omitted a final closing bracket or brace.

This could create problems for the pipeline, because if the Gap Agent output could not be parsed correctly, the parsed result might become empty or remain available only as raw text. As a result, the Decision Agent would receive incomplete intermediate information, which could negatively affect the final classification.

To make the pipeline more robust, I implemented a more flexible JSON parsing function. The parser first attempts to read the model output directly as JSON. If this fails, it searches inside the generated text for a JSON object and tries to parse the largest possible valid part. If the extracted JSON still appears incomplete, the helper function `_close_json` attempts to repair it by closing missing quotes, lists, or dictionaries.



If all parsing attempts fail, the pipeline does not crash. Instead, the original model output is stored under the raw field. This makes the system more robust to malformed model outputs and also makes debugging easier, since the original response is still preserved.

```
=== GAP EVASION RAW ===
'{"coverage": "mostly_complete", "gap_level": "minor", "best_evasion_category": "explicit_answer", "missing_requirements": ["Specific mechanism of financial assistance"], "evidence_for_answering": ["The answer explicitly states: 'Yes, the user will ensure that the Federal Government will continue to pay the insurance premiums to the affected individuals.'"], "evidence_for_evasion_or_gap": ["The answer provides a direct affirmative response, but the mechanism of how the money is actually being used (e.g., specific state programs, direct payment to individuals) is not detailed."], "gap_explanation": "The answer provides a direct affirmative answer to the 'guarantee' aspect, but lacks the specific details on the mechanism of financial assistance."}
```

**Decision Agent Output** The Decision Agent is the final stage of the pipeline. It receives the target question, the original answer, and the outputs of the previous agents, and then assigns the final clarity label.

In this example, the Decision Agent produces the wrong label. It predicts Clear Reply with confidence 0.95, while the correct gold label is Ambivalent. This confidence is too high because the answer does not clearly guarantee that federal financial assistance will continue.

The wrong decision is mainly caused by error propagation from the previous agents. The Answer Content Agent treated the final “Yes” as a clear direct answer and did not mark much of the response as off-target. Then, the Gap and Evasion Agent also treated the answer as mostly complete and explicit. As a result, the Decision Agent followed this incorrect intermediate evidence and classified the answer as a clear reply.

```
=== DECISION PARSED JSON ===
{
 "label": "Clear Reply",
 "confidence": 0.95,
 "rationale": "The answer directly and explicitly confirms the speaker's intention to ensure that people will still receive help from the Federal Government to pay their insurance.",
 "used_rule": "The answer provides a direct affirmative response to the target question, resolving the 'yes/no' intent without ambiguity or refusal."
}
```

Overall, this example shows how the agents work together and how the output of one stage can influence the next. The wrong label is not caused only by the Decision Agent. Instead, the error starts earlier, when the Answer Content Agent treats the final “Yes” as a clear answer and ignores much of the off-target content. The Gap and Evasion Agent then relies on this incomplete interpretation and treats the answer as mostly complete. As a result, the Decision Agent receives misleading evidence and predicts Clear Reply instead of Ambivalent.

This means that a mistake in one stage can affect the whole pipeline. For this reason, I tested different pipeline variants and improvements, aiming to reduce error propagation and improve the final performance of the model.

## 6.2. Implementation Details

Because the full set of experiments was computationally expensive, the experiments were divided into two groups. The first group used a validation subset of 300 samples and included the full D3 pipeline, the prompt formulation experiments, and the main ablation experiments. The second group used a smaller validation subset of 100 samples, because the KNN few-shot and DSPy optimization experiments were more time-consuming.

**Full D3 Pipeline, Prompt Formulation and Ablation Experiments** The experiments were built around a single reusable execution framework rather than a separate loop for each prompting variant. The full D3 pipeline, the prompt formulation variants, and the ablation experiments were run through the same batched runner, using different configuration settings. The full D3 experiment corresponds to running the runner with all agents enabled.

All experiments used the instruction-tuned Qwen/Qwen3.5-0.8B model from the Hugging Face transformers library, without fine-tuning. The model was therefore used purely as a prompting-based classifier. Each input was formatted as a chat conversation with a system prompt and a user message. This conversation was converted into a single prompt string and passed to the model. Only the newly generated tokens were decoded, so each output contained only the model's generated answer.

The D3 pipeline consists of four agents: a Question Intent Agent, an Answer Content Agent, a Gap and Evasion Agent, and a Decision Agent. Each agent builds its own chat prompt. The three intermediate agents are instructed to return JSON objects. These outputs are parsed by a tolerant extractor: it first tries to parse the whole output directly, then removes possible Markdown code fences and searches for a JSON object inside the generated text. If parsing still fails, the raw string is preserved. This ensures that malformed generations do not break the pipeline and can still be inspected afterwards.

Greedy decoding was used throughout the experiments. The token budget was adjusted according to the expected length of each agent output. In the main full D3 run, the Question Intent Agent used 250 new tokens, the Answer Content Agent used 450 new tokens, the Gap and Evasion Agent used 600 new tokens, and the Decision Agent used 200 new tokens.

The main runner processes the validation set stage by stage rather than example by example. It first runs the Question Intent Agent over all examples in batches, then the Answer Content Agent, then the Gap and Evasion Agent using the previous outputs, and finally the Decision Agent, which produces the final prediction. This stage-wise batching makes the experiments faster and allows the same runner to be reused across different ablation and prompt formulation settings.

The ablations are controlled by three Boolean flags: `use_question_intent`, `use_answer_content`, and `use_gap_evasion`. When all three are set to `True`, the runner executes the full D3 pipeline. When one of them is set to `False`, the corresponding agent output is omitted from the downstream context. The Decision Agent always runs, because it produces



the final clarity label. Therefore, the full pipeline, the prompt formulation variants, and the ablation experiments share the same execution framework.

**KNN Few-Shot Experiment** The KNN few-shot variant augments the Decision Agent’s prompt with retrieved examples from the training set. Each question–answer pair is embedded with the all-MiniLM-L6-v2 sentence transformer into a normalized vector, and the training pool is pre-encoded in the same embedding space. At inference time, the three nearest neighbours are retrieved using cosine similarity. In this experiment,  $k=3$  was used with an unbalanced retrieval setting.

The retrieved examples, including their question, answer, and gold label, are formatted into the Decision Agent’s prompt as calibration examples. The three intermediate agents run in the same way as in the base D3 pipeline; only the Decision Agent receives the additional few-shot context. Greedy decoding was also used in this setting. The Question Intent Agent used 250 new tokens, the Answer Content Agent used 450 new tokens, the Gap and Evasion Agent used 450 new tokens, and the Decision Agent used 150 new tokens.

**DSPy Prompt Optimization** Two agents were individually optimized with DSPy: the Gap and Evasion Agent and the Decision Agent. For each optimization run, a class-balanced training set was sampled from the training partition with a fixed random seed of 42. The upstream agent outputs were pre-computed and stored in the DSPy examples, so that the optimizer changed only the target agent being optimized.

The Gap and Evasion Agent was optimized with BootstrapFewShot using 12 training examples, with 4 examples per class. The optimizer used at most 4 bootstrapped demonstrations, no raw labeled demonstrations, one optimization round, and a maximum error tolerance of 30 generation errors. Its metric was downstream-based: for each candidate Gap and Evasion prompt, the Decision Agent was run afterwards, and the candidate was scored according to whether the final predicted label matched the gold label.

For the Decision Agent, two DSPy optimization strategies were applied. First, BootstrapFewShot was used with the same 12-example class-balanced training set and up to 4 bootstrapped demonstrations. Second, MIPROv2 was applied in light mode on the same training set, with up to 3 bootstrapped demonstrations and 2 raw labeled demonstrations. In the MIPROv2 setting, both the proposal model and the task model were the local Qwen model.

### 6.3. Output Parsing, Invalid Outputs and Fallbacks

Since the model is generative, the raw output cannot be used directly as a classification label. For this reason, a label-normalisation step was implemented. The Decision Agent output is first parsed as JSON. The value of the label field is then passed to the function `normalize_prediction_label`. This function maps the model output to one of the three official labels: *Clear Reply*, *Ambivalent*, or *Clear Non-Reply*.

If the label field is missing, or if the generated label cannot be mapped to one of the three valid labels, the fallback prediction is *Ambivalent*. This fallback was chosen because *Ambivalent* is the safest label for uncertain or partially valid cases.

## 7. Experimental Results

### 7.1. First set of Experiments

**7.1.1. Baseline 1.** As a baseline experiment, the full D3 pipeline was evaluated on the valid\_df validation subset of 300 examples. In this setting, all four agents were enabled: the Question Intent Agent, the Answer Content Agent, the Gap and Evasion Agent, and the Decision Agent.

The full D3 baseline achieves a weighted F1-score of 0.53 and a macro F1-score of 0.41. The best results are obtained for the Ambivalent class, with an F1-score of 0.67 and a recall of 0.75. In contrast, performance is weaker for Clear Reply and especially for Clear Non-Reply, whose recall is only 0.10. This shows that the model rarely identifies non-replies correctly and often prefers the middle Ambivalent category.

| Full D3         |           |        |          |         |
|-----------------|-----------|--------|----------|---------|
|                 | precision | recall | f1-score | support |
| Clear Reply     | 0.42      | 0.36   | 0.39     | 92      |
| Ambivalent      | 0.61      | 0.75   | 0.67     | 177     |
| Clear Non-Reply | 0.43      | 0.10   | 0.16     | 31      |
| accuracy        |           |        | 0.56     | 300     |
| macro avg       | 0.49      | 0.40   | 0.41     | 300     |
| weighted avg    | 0.54      | 0.56   | 0.53     | 300     |

Figure 18: Classification Report Baseline

The confusion matrix confirms this pattern: most Ambivalent examples are classified correctly, while many true Clear Reply and Clear Non-Reply examples are also predicted as Ambivalent.

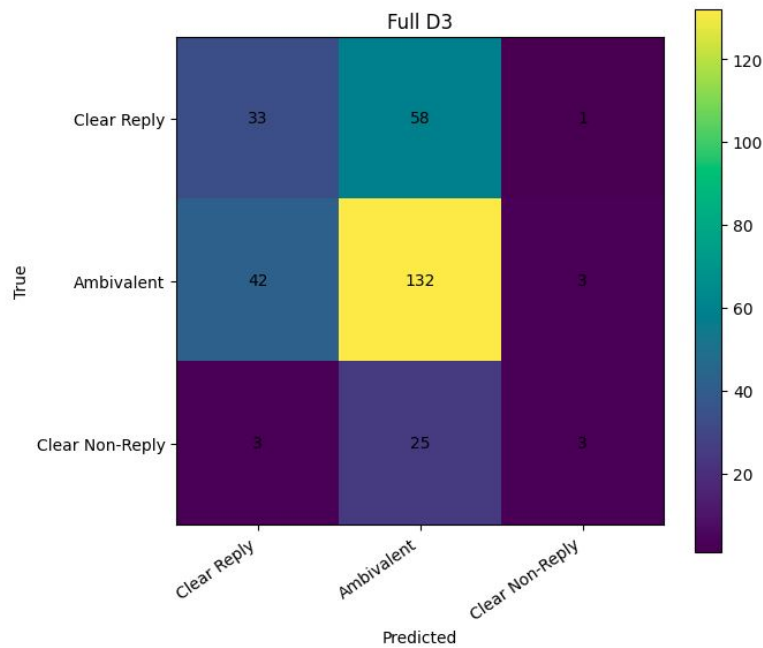


Figure 19: Confusion Matrix Report Baseline

### 7.1.2. Ablation Experiments.

**No Gap/Evasion Agent** In this ablation experiment, the Gap and Evasion Agent was removed from the pipeline. The downstream prompts were modified so that they no longer depended on the gap analysis output. In this setting, the Decision Agent directly compares the Question Intent output, the Answer Content output, and the original answer in order to assign the final clarity label.

|                      |           |        |          |         |
|----------------------|-----------|--------|----------|---------|
| =====                |           |        |          |         |
| D3 without Gap Agent |           |        |          |         |
| =====                |           |        |          |         |
|                      | precision | recall | f1-score | support |
| Clear Reply          | 0.42      | 0.68   | 0.52     | 92      |
| Ambivalent           | 0.63      | 0.52   | 0.57     | 177     |
| Clear Non-Reply      | 0.60      | 0.10   | 0.17     | 31      |
| accuracy             |           |        | 0.53     | 300     |
| macro avg            | 0.55      | 0.43   | 0.42     | 300     |
| weighted avg         | 0.56      | 0.53   | 0.51     | 300     |

Figure 20: Classification Report No Gap/Evasion Agent

Removing the Gap and Evasion Agent changes the behaviour of the pipeline. The overall accuracy decreases from 0.56 to 0.53, and the weighted F1-score decreases from 0.53 to 0.51. The macro F1-score remains very similar, increasing slightly from 0.41 to 0.42. However, this small macro-F1 difference hides an important change in the class-level behaviour.

Without the Gap and Evasion Agent, the recall for Clear Reply increases substantially, from 0.36 to 0.68. This means that the model predicts Clear Reply much more

often when the gap-analysis stage is removed. At the same time, the recall for Ambivalent decreases from 0.75 to 0.52. Therefore, many examples that were previously classified as Ambivalent are now shifted toward Clear Reply. The performance on Clear Non-Reply remains very weak, with an F1-score of only 0.17.

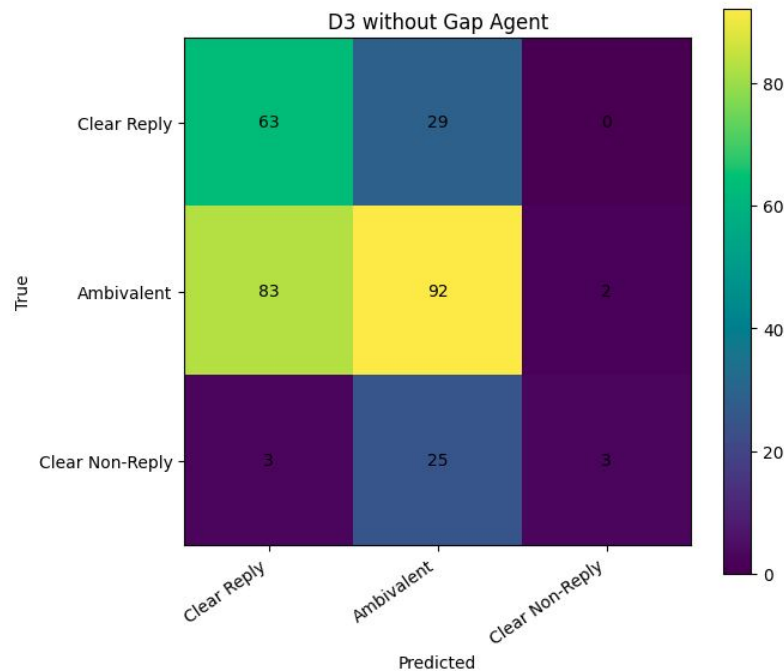


Figure 21: Confusion matrix No Gap/Evasion Agent

The confusion matrix shows this shift clearly. In the full D3 pipeline, 58 true Clear Reply examples were misclassified as Ambivalent. After removing the Gap and Evasion Agent, this number decreases to 29, while the number of correctly classified Clear Reply examples increases from 33 to 63. However, this improvement comes at the cost of worse performance on the Ambivalent class. The number of true Ambivalent examples predicted as Clear Reply increases from 42 to 83.

This suggests that the Gap and Evasion Agent helps the pipeline remain more cautious when an answer is only partially responsive. Without this agent, the Decision Agent becomes more likely to treat partially relevant answers as clear replies.

**Question and Answer in One Prompt** In this prompt formulation experiment, the Question Intent Agent and the Answer Content Agent were merged into a single prompt. Instead of first analysing the target question and then separately extracting the answer content, the merged agent receives both the question and the answer together and produces a combined intermediate analysis. The Gap and Evasion Agent and the Decision Agent are then applied as in the full D3 pipeline.

For the same example used in the baseline analysis, the merged Question–Answer Agent produces a combined interpretation of the target question and the answer. The output shows that the agent correctly identifies the general focus of the question, namely whether the speaker will ensure continued federal help for insurance payments. It also extracts relevant answer content related to affordable health care and

identifies some general or filler content, such as the criticism of Obamacare and the references to different states.

However, the merged agent still treats the answer as more explicit than it really is. It sets `explicit_response_present` to true and summarizes the answer as explicitly stating that the speaker is looking to ensure continued federal help. This repeats the same weakness observed in the full D3 pipeline: the final affirmative tone of the answer is interpreted as a clear response, while the broader context remains indirect and incomplete. In addition, the agent includes affordability and coverage gaps in `must_address`, although the target question mainly asks whether federal financial help will continue. Therefore, the merged prompt captures useful information, but it does not fully resolve the ambiguity of the answer.

```

=== PARSED JSON ===
{
 "question_focus": "Are you going to ensure that those people will still get
 help from the Federal Government to pay that insurance?",
 "expected_answer_type": "yes",
 "must_address": [
 "ensure federal government payment for insurance",
 "address the issue of affordability and coverage gaps"
],
 "explicit_response_present": true,
 "explicit_response_summary": "The answer explicitly states that the speaker
 is looking to ensure that people will still get help from the Federal
 Government to pay that insurance.",
 "relevant_answer_content": [
 "We want to get it down so that people can have affordable health care.",
 "We want to get health care that's much more affordable and much better
 health care."
],
 "concrete_details": [
 "116 percent up",
 "over 200 percent up",
 "50 percent, 70 percent up",
 "States that are doing better",
 "States that are doing worse"
],
 "general_or_filler_content": [
 "Obamacare is a wreck, it's a mess, it's destroying lives.",
 "We want to get it in those States the States that I did so well in but also
 in States that I didn't win."
]
}

```

This variant achieves a weighted F1-score of 0.49 and a macro F1-score of 0.36, which is lower than the full D3 baseline. The Ambivalent class remains the best-performing class, with an F1-score of 0.60 and recall of 0.60. However, performance on Clear Non-Reply becomes very weak, with a recall of only 0.03 and an F1-score of 0.06. This shows that merging the question and answer analysis does not help the model identify clear non-replies.

| D3 with Merged Q-A Agent |           |        |          |         |
|--------------------------|-----------|--------|----------|---------|
|                          | precision | recall | f1-score | support |
| Clear Reply              | 0.35      | 0.48   | 0.41     | 92      |
| Ambivalent               | 0.61      | 0.60   | 0.60     | 177     |
| Clear Non-Reply          | 0.50      | 0.03   | 0.06     | 31      |
| accuracy                 |           |        | 0.50     | 300     |
| macro avg                | 0.49      | 0.37   | 0.36     | 300     |
| weighted avg             | 0.52      | 0.50   | 0.49     | 300     |

Figure 22: Classification report for D3 with the merged Question–Answer Agent

The confusion matrix shows that the model still predicts many examples as Ambivalent. However, compared with the full D3 pipeline, more true Ambivalent examples are incorrectly predicted as Clear Reply. In addition, almost all true Clear Non-Reply examples are misclassified, mostly as Ambivalent or Clear Reply. Therefore, this prompt formulation weakens the separation between partially responsive answers and non-replies.

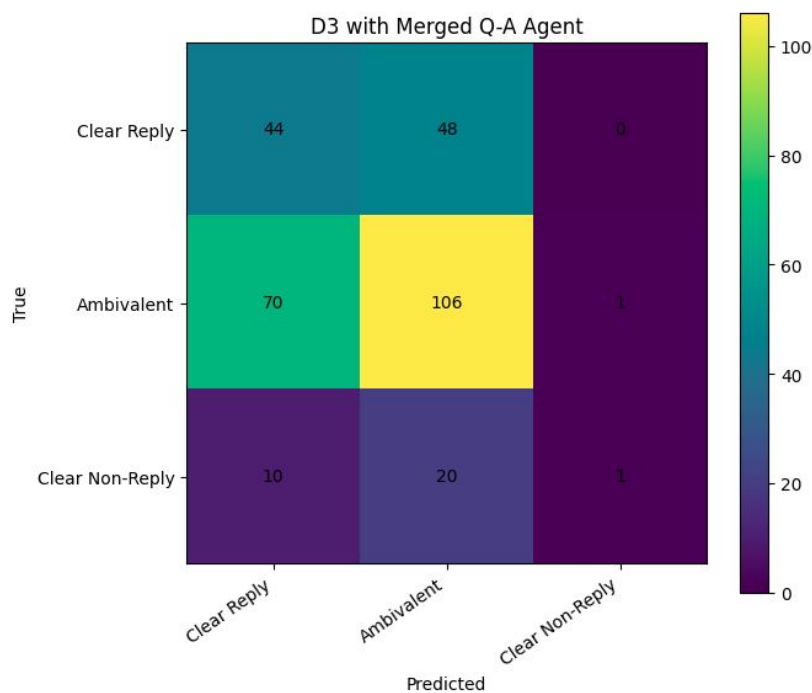


Figure 23: Confusion matrix for D3 with the merged Question–Answer Agent

### 7.1.3. Prompt Formulation.

**Revised Question Intent Agent** In this prompt formulation experiment, only the Question Intent Agent prompt was revised, while the rest of the D3 pipeline remained unchanged. The goal of this change was to make the first agent produce a cleaner interpretation of the target question, especially by avoiding placeholder fields and by selecting a single question type. The revised schema was also made slightly more

compact: the `must_address` field, which in the original prompt was the one most often filled with placeholder requirements, was removed, and the instruction block was condensed (the longer dataset-rule note and the requirement bullets were shortened), so the agent now returns a smaller JSON object focused on the question type, intent, clear-answer criteria, and minimal sufficient answer.

```
QUESTION_INTENT_PROMPT_Revised = """
You are the Question Intent Agent for a response-clarity classification task.

Analyze ONLY the TARGET_QUESTION. Identify exactly what information
would make an answer clear.

Think in terms of answer requirements:
- Is the question asking for yes/no?
- Is it asking what / why / how something happened?
- Is it asking for a specific action, plan, reason, stance, or evaluation?

Return exactly one JSON object and nothing else.

JSON schema:
{
 "question_type": "yes_no|what|why|how|when|where|who|stance|explanation|
 action_plan|evaluation|other",
 "question_intent": "one sentence describing what the target question asks",
 "clear_answer_criteria": "one sentence describing what a clear answer must
 contain",
 "minimal_sufficient_answer": "minimum information needed for Clear Reply"
}
"""
```

For the running example, the revised Question Intent Agent produces a cleaner output than the original version. It correctly assigns the question type as `yes_no` and reformulates the question intent as a commitment to ensuring continued federal financial assistance. It also provides a clearer answer criterion and a minimal sufficient answer. This shows that the revised prompt improves the structure of the Question Intent output.

```
=== QUESTION INTENT PARSED JSON ===
{
 "question_type": "yes_no",
 "question_intent": "Are you committed to ensuring that the Federal Government
 will continue to provide financial assistance to those individuals?",
 "clear_answer_criteria": "The answer must explicitly state whether the speaker
 is going to or not going to ensure that the Federal Government will pay
 the insurance.",
 "minimal_sufficient_answer": "Yes, I am going to ensure that the Federal
 Government will pay the insurance."
}
```



| Full D3 Revised Question Agent |           |        |          |         |
|--------------------------------|-----------|--------|----------|---------|
|                                | precision | recall | f1-score | support |
| Clear Reply                    | 0.40      | 0.39   | 0.40     | 92      |
| Ambivalent                     | 0.61      | 0.70   | 0.65     | 177     |
| Clear Non-Reply                | 0.44      | 0.13   | 0.20     | 31      |
| accuracy                       |           |        | 0.55     | 300     |
| macro avg                      | 0.49      | 0.41   | 0.42     | 300     |
| weighted avg                   | 0.53      | 0.55   | 0.53     | 300     |

Figure 24: Classification report for the full D3 pipeline with the revised Question Intent Agent

The revised Question Intent Agent achieves a weighted F1-score of 0.53 and a macro F1-score of 0.42. Compared with the full D3 baseline, the weighted F1-score remains the same, while the macro F1-score improves slightly from 0.41 to 0.42. This suggests that improving the structure of the Question Intent output helps only marginally at the final classification level.

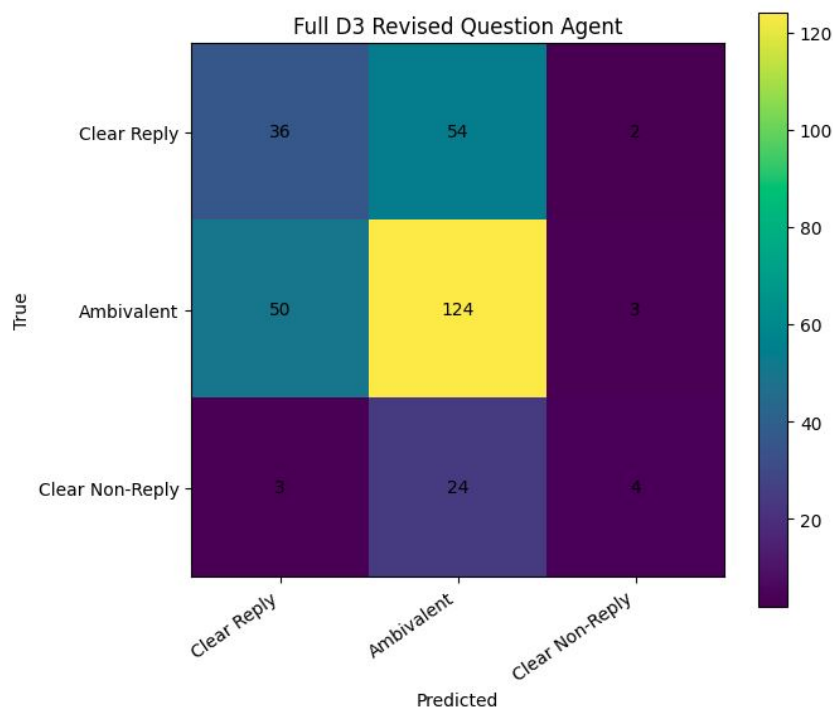


Figure 25: Confusion matrix for the full D3 pipeline with the revised Question Intent Agent

The confusion matrix shows a pattern similar to the baseline. Most Ambivalent examples are still classified correctly, while many true Clear Reply and Clear Non-Reply examples are still predicted as Ambivalent. However, there is a small improvement for Clear Non-Reply, with 4 correct predictions instead of 3 in the baseline. Overall, the revised Question Intent prompt improves the intermediate output quality, but it does not substantially change the final prediction behaviour of the pipeline.

**Revised Answer Agent** In this prompt formulation experiment, only the Answer Content Agent prompt was revised, while the rest of the D3 pipeline remained unchanged. The goal of this change was to make the answer analysis more precise and less over-optimistic. The revised prompt applies a stricter definition of a direct answer: a response counts as direct only when it explicitly states yes or no, and any additional content must actually address the target question, otherwise the response is not treated as a direct answer. In addition, the schema was simplified: the over-positive `relevant_answer_summary` and the separate `vague_or_general_information` fields were removed, vague, rhetorical, and filler statements are now folded into `off_target_content` and a new `stated_position` field was added to capture the speaker's explicit position. A short worked example was also included to calibrate the agent.

```
ANSWER_CONTENT_PROMPT_Revised = ""
You are the Answer Content Agent for a response-clarity classification task.

Important dataset rule:
The answer may respond to a longer interview question, but you must evaluate
 what it says with respect to the TARGET_QUESTION only.

Your task:
Extract what the answer actually says that is relevant to the TARGET_QUESTION.

Do not decide the final label.
Do not reward political rhetoric unless it answers the target question.
Do not infer facts that are not present in the answer.
A response counts as a direct answer only if it explicitly states yes or no.
If it adds further information, that information must actually address the
question; if the extra content is off-topic or general, the response does NOT
count as a direct answer.

Look for:
concrete facts, actions, reasons, explanations, examples, commitments
vague but related statements
topic shifts
refusals or declines to answer
answers to a different sub-question

JSON schema:
{
 "direct_answer_present": true,
 "direct_answer_text": "exact short phrase from the answer, or empty string",
 "concrete_information": ["specific on-topic point actually stated in the
 answer"],
 "off_target_content": ["vague, off-topic, rhetorical, or filler part"],
 "refusal_or_non_answer_signals": ["signal that the speaker refuses/declines,
 else empty list"],
 "stated_position": "the speaker's explicitly stated position, or 'none'"
}

Example:
```

```

TARGET_QUESTION: Will you raise the minimum wage?
ANSWER: Look, the previous government destroyed this economy, it was a total
 disaster.
We care about working families more than anyone ever has. Yes, we will raise
 it.
Output:
{
 "direct_answer_present": true,
 "direct_answer_text": "Yes, we will raise it.",
 "concrete_information": ["The speaker commits to raising the minimum wage
 ."],
 "off_target_content": ["Attack on the previous government ('destroyed this
 economy', 'a total disaster')", "General claim about caring for working
 families"],
 "refusal_or_non_answer_signals": [],
 "stated_position": "In favor of raising the minimum wage."
}
"""

```

For the running example, the revised Answer Content Agent identifies that a direct answer is present and extracts the direct answer text as "Yes". It also separates the relevant concrete information from the more general or off-target political content. For example, it recognises that the speaker discusses affordable health care and gives numerical examples, but it also marks attacks on the previous government and general claims about the system as off-target content. This shows that the revised prompt produces a more structured description of what the answer actually contains.

```

=== ANSWER CONTENT PARSED JSON ===
{
 "direct_answer_present": true,
 "direct_answer_text": "Yes",
 "concrete_information": [
 "The speaker states that the goal is to get health care down to affordable
 levels",
 "The speaker mentions specific examples of states achieving 116 percent,
 200 percent, and 50-70 percent increases in health care access"
],
 "off_target_content": [
 "Attacking the previous government's actions",
 "General claims about the current system being a 'wreck' or 'mess'",
 "General claims about the speaker's personal success in states"
],
 "refusal_or_non_answer_signals": [],
 "stated_position": "In favor of reducing health care costs and improving
 access to affordable care."
}

```

| Full D3 Revised Answer Agent |           |        |          |         |
|------------------------------|-----------|--------|----------|---------|
|                              | precision | recall | f1-score | support |
| Clear Reply                  | 0.48      | 0.32   | 0.38     | 92      |
| Ambivalent                   | 0.63      | 0.79   | 0.70     | 177     |
| Clear Non-Reply              | 0.50      | 0.32   | 0.39     | 31      |
| accuracy                     |           |        | 0.59     | 300     |
| macro avg                    | 0.54      | 0.47   | 0.49     | 300     |
| weighted avg                 | 0.57      | 0.59   | 0.57     | 300     |

Figure 26: Classification report for the full D3 pipeline with the revised Answer Agent

The revised Answer Agent achieves an accuracy of 0.59, a weighted F1-score of 0.57, and a macro F1-score of 0.49. This is one of the strongest results in this set of experiments. The model performs particularly well on the Ambivalent class, reaching a recall of 0.79 and an F1-score of 0.70. This suggests that the revised answer analysis helps the pipeline identify partially responsive answers more effectively.

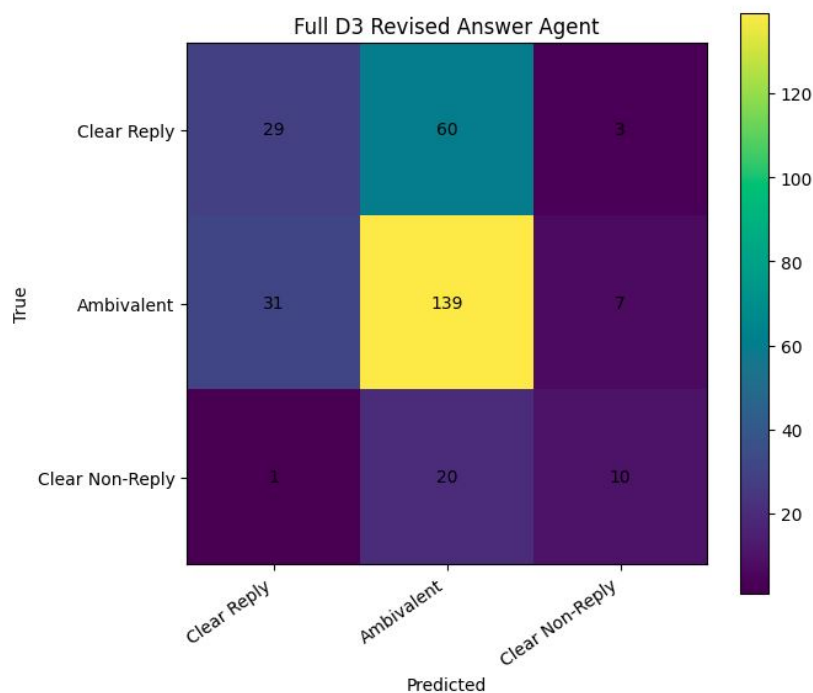


Figure 27: Confusion matrix for the full D3 pipeline with the revised Answer Agent

The confusion matrix shows that most Ambivalent examples are correctly classified, with 139 out of 177 predicted correctly. However, many true Clear Reply examples are still predicted as Ambivalent, which shows that the model remains cautious when deciding whether an answer fully resolves the target question. The Clear Non-Reply class also remains difficult, although this variant correctly predicts 10 examples from this class. Overall, the revised Answer Agent improves the quality of the intermediate answer representation and leads to better final classification performance.

**D3 Gap Robust to the Answer Agent** In this variant, only the Gap and Evasion Agent prompt was revised, while the rest of the D3 pipeline remained unchanged. The motivation came from the error analysis of the baseline, where the Answer Content Agent often produced an over-optimistic extraction (for example treating a trailing “Yes” as a direct answer), and this error then propagated to the Gap and Evasion Agent. The goal of this variant was therefore to make the gap analysis more robust to the Answer Content Agent: instead of trusting the upstream extraction, the Gap and Evasion Agent was instructed to re-check the coverage directly against the original answer and to treat the Answer Content output only as a secondary signal, so that an over-positive extraction would no longer automatically lead to a small gap.

For the running example, however, the gap-robust agent still rates the coverage as `mostly_complete` with only a minor gap and labels the response as an `explicit_answer`. Even when instructed to re-check the raw answer, it does not detect the missing guarantee on this case, which already foreshadows the limited effect of this change.

```
=== GAP EVASION PARSED JSON ===
{
 "coverage": "mostly_complete",
 "gap_level": "minor",
 "best_evasion_category": "explicit_answer",
 "missing_requirements": [
 "specific requirement 2"
],
 "evidence_for_answering": [
 "The answer explicitly states: 'Yes, I am going to ensure that those people will still get help from the Federal Government to pay that insurance.'"
],
 "evidence_for_evasion_or_gap": [
 "The answer provides a direct affirmative response to the core question, but the specific requirement 'specific requirement 2' (the exact phrasing of the question) is not fully captured in the summary or the direct answer text."
],
 "gap_explanation": "The answer provides a direct affirmative response to the core question, but the specific requirement 'specific requirement 2' is not fully captured in the summary or the direct answer text."
}
```

| D3 Gap robust to Answer Agent |           |        |          |         |
|-------------------------------|-----------|--------|----------|---------|
|                               | precision | recall | f1-score | support |
| Clear Reply                   | 0.39      | 0.61   | 0.47     | 92      |
| Ambivalent                    | 0.62      | 0.52   | 0.57     | 177     |
| Clear Non-Reply               | 0.57      | 0.13   | 0.21     | 31      |
| accuracy                      |           |        | 0.51     | 300     |
| macro avg                     | 0.53      | 0.42   | 0.42     | 300     |
| weighted avg                  | 0.54      | 0.51   | 0.50     | 300     |

Figure 28: Classification report for the full D3 pipeline with the Gap Agent made robust to the Answer Agent

This variant achieves an accuracy of 0.51, a weighted F1-score of 0.50, and a macro F1-score of 0.42. The Ambivalent class remains the best-performing one (F1-score 0.57), while Clear Reply reaches an F1-score of 0.47 and Clear Non-Reply remains the hardest class, with an F1-score of only 0.21. Compared with the full D3 baseline, the weighted F1-score drops from 0.53 to 0.50, so re-grounding the gap analysis on the raw answer does not improve the overall performance.

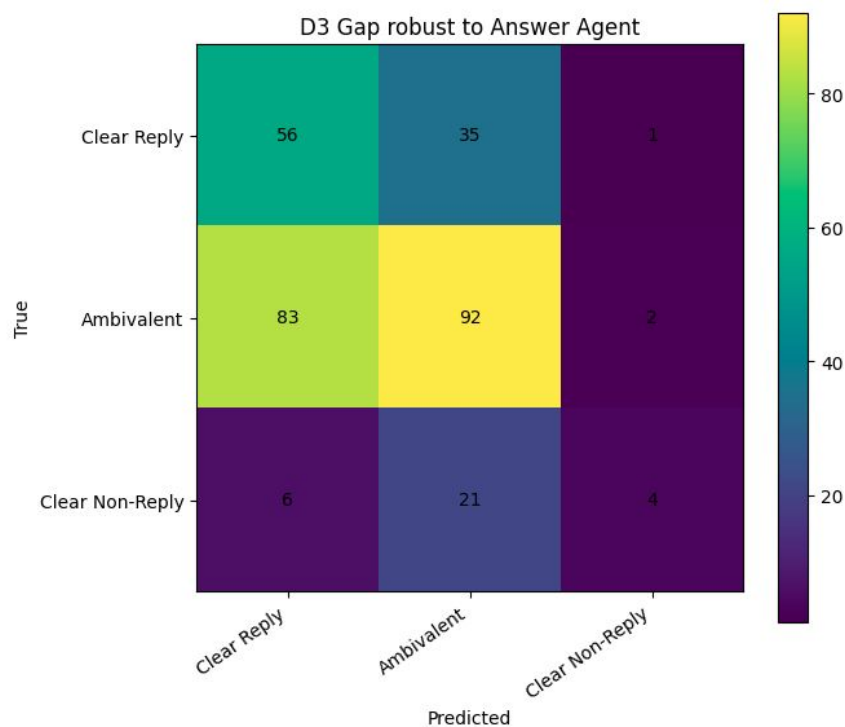


Figure 29: Confusion matrix for the full D3 pipeline with the Gap Agent made robust to the Answer Agent

The confusion matrix explains this drop. The recall for Clear Reply increases sharply (56 of 92 correct, versus 33 in the baseline), but this comes at the expense of the Ambivalent class: 83 true Ambivalent examples are now predicted as Clear Reply, and the Ambivalent recall falls to 0.52. In other words, by making the Gap and Evasion Agent less dependent on the Answer Content Agent, the pipeline loses the cautious

behaviour that previously kept borderline answers in the middle category, and it starts to over-predict Clear Reply. This is very similar to the effect of removing the Gap and Evasion Agent altogether (Section 7.1.2). The minority Clear Non-Reply class barely changes, with only 4 of 31 examples correctly classified. Overall, this variant confirms that the value of the Gap and Evasion Agent lies precisely in comparing the question against the extracted answer content: when this coupling is weakened, the pipeline becomes over-confident and the weighted F1-score decreases.

## 7.2. Second Set of Experiments

The second set of experiments was evaluated on a smaller validation subset of 100 examples. This was done because the KNN few-shot and DSPy optimization experiments were more computationally expensive than the previous prompt formulation and ablation experiments. Therefore, the results in this section are mainly compared with the corresponding baseline on the same 100-example subset.

**7.2.1. Baseline 2.** As a baseline for the second experimental setting, the full D3 pipeline was first evaluated on the 100-example validation subset. This baseline uses the same four-agent structure as before: Question Intent Agent, Answer Content Agent, Gap and Evasion Agent, and Decision Agent.

```

=====
Full D3
=====

```

|                 | precision | recall | f1-score | support |
|-----------------|-----------|--------|----------|---------|
| Clear Reply     | 0.35      | 0.29   | 0.32     | 31      |
| Ambivalent      | 0.58      | 0.73   | 0.65     | 59      |
| Clear Non-Reply | 0.00      | 0.00   | 0.00     | 10      |
| accuracy        |           |        | 0.52     | 100     |
| macro avg       | 0.31      | 0.34   | 0.32     | 100     |
| weighted avg    | 0.45      | 0.52   | 0.48     | 100     |

Figure 30: Classification report for the full D3 baseline on the second validation subset

The baseline achieves a weighted F1-score of 0.48 and a macro F1-score of 0.32. As in the previous experiments, the best-performing class is Ambivalent, with an F1-score of 0.65 and a recall of 0.73. However, the model performs poorly on the two more decisive classes. In particular, Clear Non-Reply receives an F1-score of 0.00, showing that the baseline does not correctly identify this class in the 100-example subset.



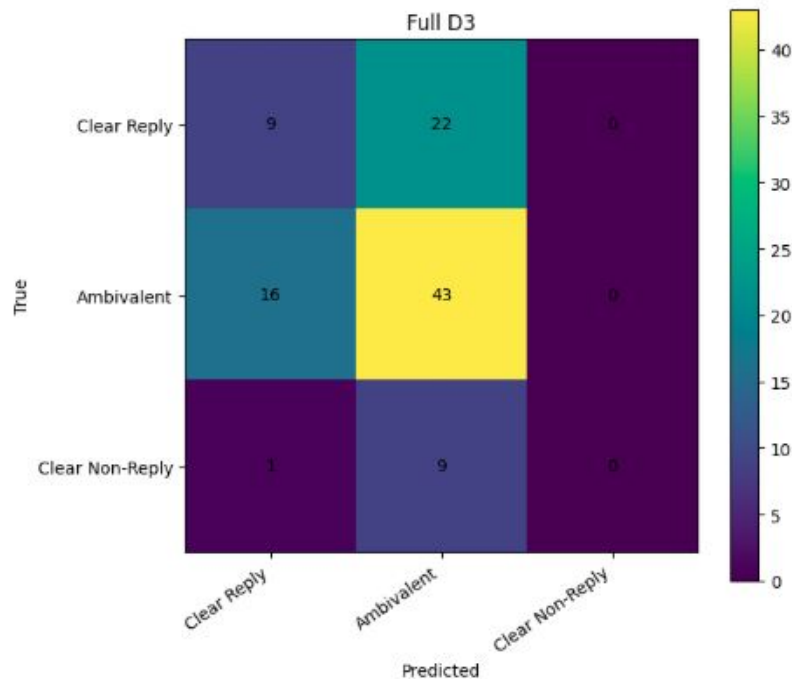


Figure 31: Confusion matrix for the full D3 baseline on the second validation subset

The confusion matrix shows that the baseline strongly favours the Ambivalent class. Most Ambivalent examples are classified correctly, but many Clear Reply and all Clear Non-Reply examples are also predicted as Ambivalent. This confirms the same tendency observed in the first set of experiments: the pipeline often assigns uncertain or difficult cases to the middle category.

**7.2.2. KNN Few-Shot Examples for the Decision Agent.** In this experiment, the Decision Agent was augmented with retrieved few-shot examples. For each validation example, the nearest training examples were selected using sentence embeddings and cosine similarity. These examples were then inserted into the Decision Agent prompt, while the previous agents remained unchanged.

```
=====
D3 knn few shots
=====
```

|                 | precision | recall | f1-score | support |
|-----------------|-----------|--------|----------|---------|
| Clear Reply     | 0.34      | 0.32   | 0.33     | 31      |
| Ambivalent      | 0.58      | 0.66   | 0.62     | 59      |
| Clear Non-Reply | 0.50      | 0.20   | 0.29     | 10      |
| accuracy        |           |        | 0.51     | 100     |
| macro avg       | 0.48      | 0.39   | 0.41     | 100     |
| weighted avg    | 0.50      | 0.51   | 0.50     | 100     |

Figure 32: Classification report for D3 with KNN few-shot examples

The KNN few-shot variant improves the results compared with the second baseline. The weighted F1-score increases from 0.48 to 0.50, and the macro F1-score in-

creases from 0.32 to 0.41. The most important improvement appears in the Clear Non-Reply class, where the F1-score increases from 0.00 to 0.29. This suggests that adding retrieved examples helps the Decision Agent better calibrate its final label choices.

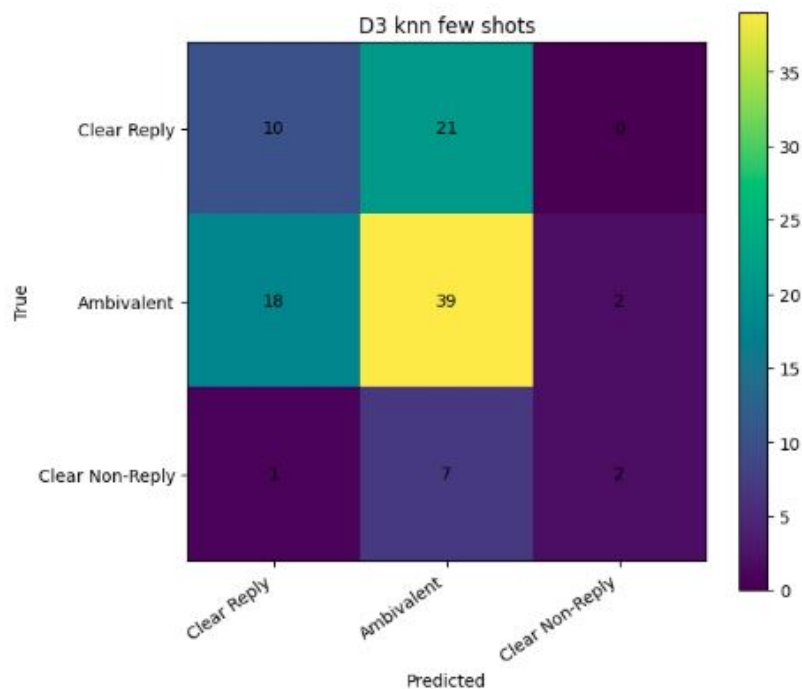


Figure 33: Confusion matrix for D3 with KNN few-shot examples

The confusion matrix shows that the model still predicts many examples as Ambivalent, but it now correctly identifies some Clear Non-Reply cases. This indicates that the retrieved examples provide useful guidance for difficult minority-class decisions, although the model still struggles to clearly separate all three classes.

### 7.2.3. DSPy Prompt Optimization.

**Optimized Gap/Evasion Agent** In this experiment, the Gap and Evasion Agent was optimized with DSPy using BootstrapFewShot. The goal was to improve the intermediate gap analysis while keeping the rest of the D3 pipeline fixed.

```
=====
D3 with Dspy Gap Agent BootstrapFewShot
=====
```

|                 | precision | recall | f1-score | support |
|-----------------|-----------|--------|----------|---------|
| Clear Reply     | 0.35      | 0.48   | 0.41     | 31      |
| Ambivalent      | 0.60      | 0.58   | 0.59     | 59      |
| Clear Non-Reply | 0.00      | 0.00   | 0.00     | 10      |
| accuracy        |           |        | 0.49     | 100     |
| macro avg       | 0.32      | 0.35   | 0.33     | 100     |
| weighted avg    | 0.46      | 0.49   | 0.47     | 100     |

Figure 34: Classification report for D3 with DSPy-optimized Gap and Evasion Agent

The DSPy-optimized Gap and Evasion Agent achieves a weighted F1-score of 0.47 and a macro F1-score of 0.33. These results are very close to the second baseline and do not show a clear improvement. The model improves the recall of Clear Reply compared with the baseline, but it still fails to correctly predict Clear Non-Reply, whose F1-score remains 0.00.

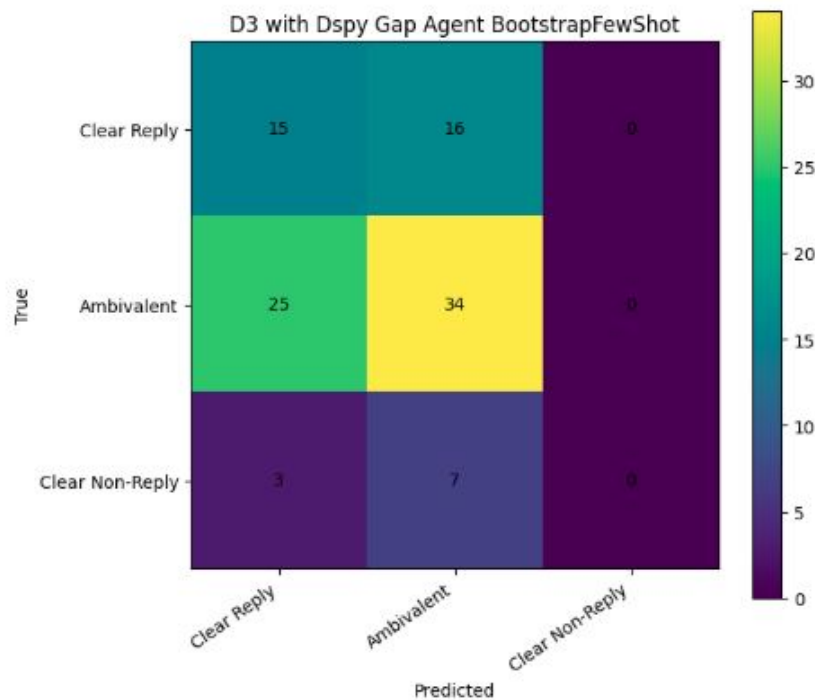


Figure 35: Confusion matrix for D3 with DSPy-optimized Gap and Evasion Agent

The confusion matrix shows that the optimized Gap Agent shifts some predictions from Ambivalent to Clear Reply. However, it does not solve the main problem of the baseline, since all Clear Non-Reply examples are still misclassified. Therefore, this optimization does not substantially improve the final classification behaviour.

**Optimized Decision Agent** The Decision Agent was also optimized with DSPy, using two optimization strategies: BootstrapFewShot and MIPROv2. Since the Decision Agent is responsible for the final label, this experiment examines whether optimizing the final decision prompt can improve the classification results.

| D3 with Dspy Decision Agent BootstrapFewShot |           |        |          |         |
|----------------------------------------------|-----------|--------|----------|---------|
|                                              | precision | recall | f1-score | support |
| Clear Reply                                  | 0.39      | 0.55   | 0.45     | 31      |
| Ambivalent                                   | 0.50      | 0.03   | 0.06     | 59      |
| Clear Non-Reply                              | 0.17      | 0.90   | 0.29     | 10      |
| accuracy                                     |           |        | 0.28     | 100     |
| macro avg                                    | 0.35      | 0.49   | 0.27     | 100     |
| weighted avg                                 | 0.43      | 0.28   | 0.21     | 100     |

Figure 36: Classification report for D3 with DSPy Decision Agent BootstrapFewShot

The BootstrapFewShot-optimized Decision Agent changes the model behaviour strongly. It improves the recall of Clear Non-Reply to 0.90, but the overall accuracy drops to 0.28 and the weighted F1-score drops to 0.21. This means that the model becomes much more willing to predict Clear Non-Reply, but this comes at the cost of many incorrect predictions for the other classes.

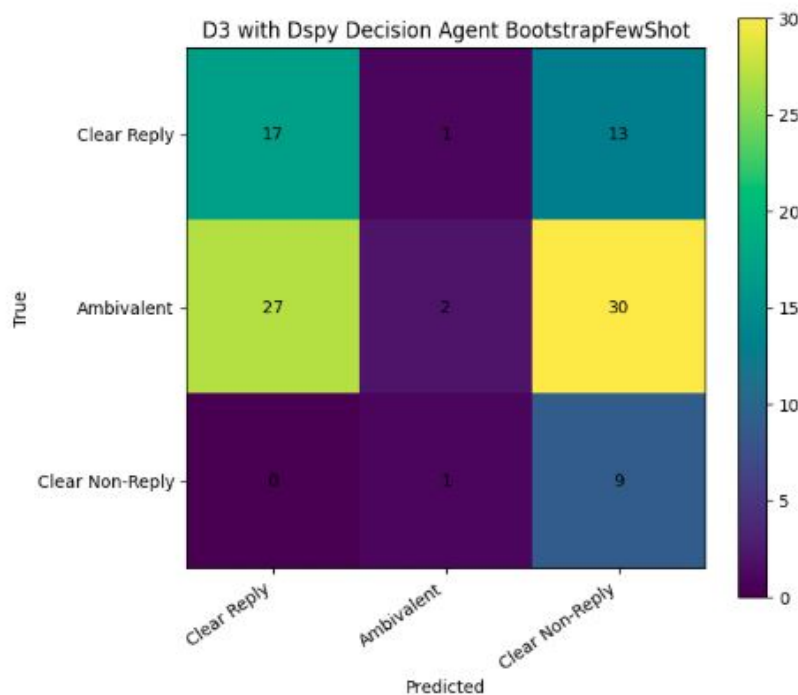


Figure 37: Confusion matrix for D3 with DSPy Decision Agent BootstrapFewShot

The confusion matrix confirms this overcorrection. Many Ambivalent examples are incorrectly predicted as Clear Reply or Clear Non-Reply, while very few are correctly classified. Therefore, although the optimization improves the recognition of non-replies, it damages the overall balance of the classifier.

| D3 with Dspy Decision Agent MIPROv2 |           |        |          |         |
|-------------------------------------|-----------|--------|----------|---------|
|                                     | precision | recall | f1-score | support |
| Clear Reply                         | 0.34      | 0.61   | 0.44     | 31      |
| Ambivalent                          | 0.54      | 0.37   | 0.44     | 59      |
| Clear Non-Reply                     | 0.33      | 0.10   | 0.15     | 10      |
| accuracy                            |           |        | 0.42     | 100     |
| macro avg                           | 0.40      | 0.36   | 0.34     | 100     |
| weighted avg                        | 0.46      | 0.42   | 0.41     | 100     |

Figure 38: Classification report for D3 with DSPy Decision Agent MIPROv2

The MIPROv2-optimized Decision Agent gives more balanced results than BootstrapFewShot. It achieves a weighted F1-score of 0.41 and a macro F1-score of 0.34. This is lower than the KNN few-shot variant, but it avoids the extreme behaviour observed with BootstrapFewShot. The recall for Clear Reply increases to 0.61, while Clear Non-Reply is still difficult, with a recall of 0.10.

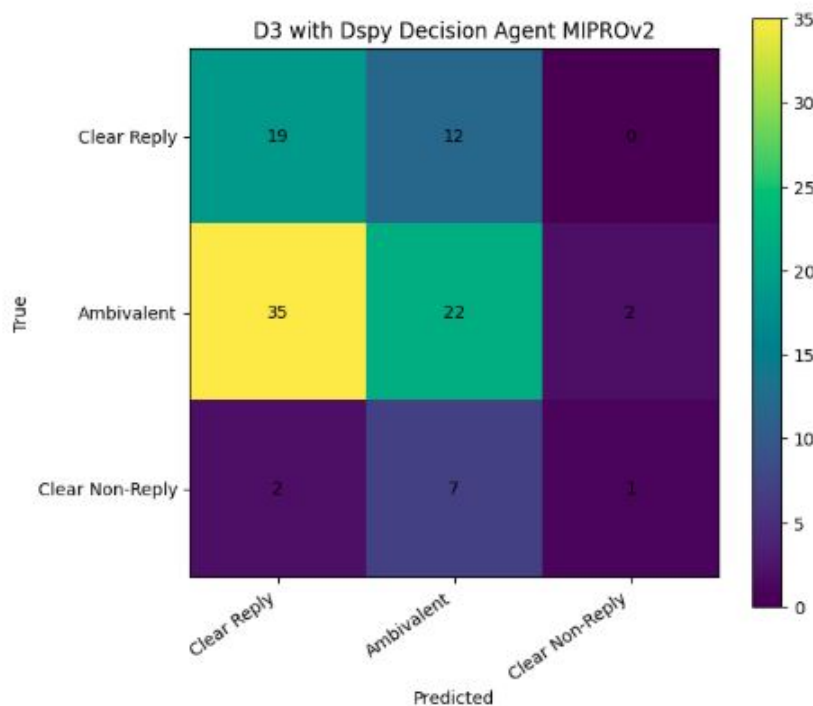


Figure 39: Confusion matrix for D3 with DSPy Decision Agent MIPROv2

The confusion matrix shows that MIPROv2 shifts more predictions toward Clear Reply, improving the number of correctly classified clear replies. However, many Ambivalent examples are also incorrectly classified as Clear Reply. Overall, the second set of experiments shows that KNN few-shot prompting provides the most useful improvement among these variants, while DSPy optimization changes the prediction behaviour but does not consistently improve the overall performance.

### 7.3. Overall Results

**7.3.1. First Set.** The first set of experiments was evaluated on the validation subset of 300 examples. It includes the full D3 baseline, the two ablation variants (the pipeline without the Gap and Evasion Agent and the merged Question–Answer Agent), the prompt-formulation variants (the revised Question Intent Agent and the revised Answer Agent), and the gap-robust variant. Table 3 reports the main classification metrics for all variants, sorted by weighted F1-score.

Table 3: Comparison of D3 variants on the first validation subset of 300 examples (sorted by weighted F1).

| Model                                   | Acc.  | Prec. Macro | Rec. Macro | F1 Macro     | Prec. W. | Rec. W. | F1 W.        |
|-----------------------------------------|-------|-------------|------------|--------------|----------|---------|--------------|
| Full D3 – Revised Answer Agent          | 0.593 | 0.537       | 0.474      | <b>0.491</b> | 0.572    | 0.593   | <b>0.571</b> |
| Full D3                                 | 0.560 | 0.489       | 0.400      | 0.407        | 0.536    | 0.560   | 0.533        |
| Full D3 – Revised Question Intent Agent | 0.547 | 0.488       | 0.407      | 0.417        | 0.532    | 0.547   | 0.529        |
| D3 without Gap Agent                    | 0.527 | 0.551       | 0.434      | 0.420        | 0.563    | 0.527   | 0.514        |
| D3 Gap robust to Answer Agent           | 0.507 | 0.526       | 0.419      | 0.416        | 0.544    | 0.507   | 0.501        |
| D3 with Merged Q-A Agent                | 0.503 | 0.488       | 0.370      | 0.357        | 0.520    | 0.503   | 0.488        |

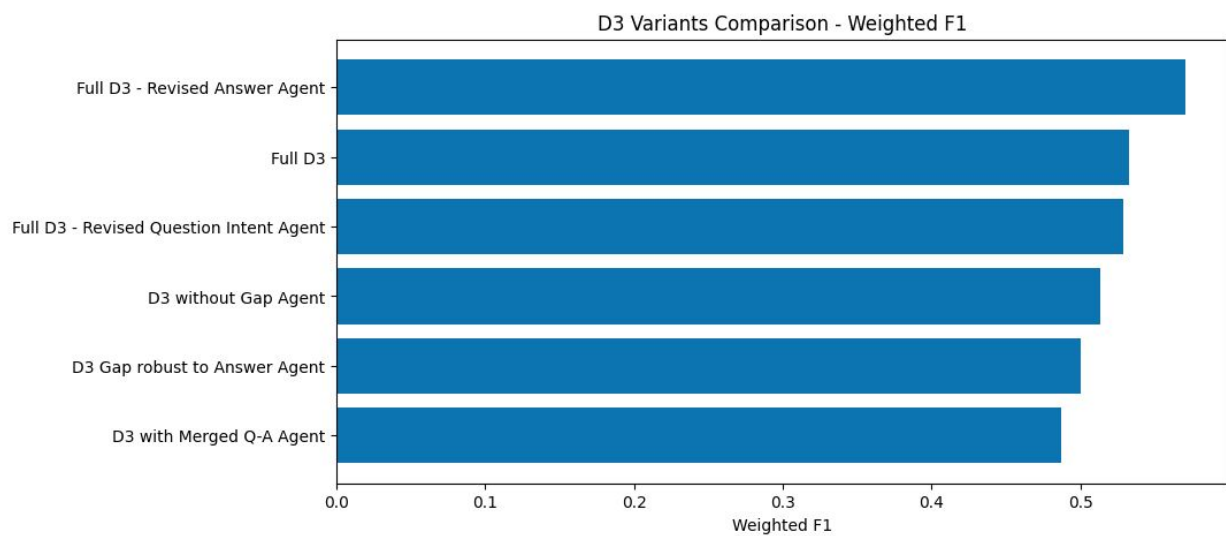


Figure 40: Weighted F1 comparison for the first set of D3 variants

The best-performing variant in this set is the full D3 pipeline with the revised Answer Agent, which reaches a weighted F1-score of 0.571 and a macro F1-score of 0.491, the highest on both metrics. This is a clear improvement over the full D3 baseline (0.533 weighted F1 and 0.407 macro F1) and confirms that making the answer analysis more conservative reduces the over-optimistic content extraction that pushes borderline answers toward Clear Reply. The revised Question Intent Agent gives a smaller improvement, mainly on the macro F1-score (0.417 vs. 0.407), while leaving the weighted F1-score close to the baseline (0.529). In contrast, the variants that simplify the pipeline perform worse: removing the Gap and Evasion Agent lowers the weighted F1-score

to 0.514, and merging the Question and Answer agents into a single prompt produces the weakest results overall (0.488 weighted F1 and 0.357 macro F1), confirming that the separation between the two analysis stages is important for the final decision.

|                                         |                      |              |                   |
|-----------------------------------------|----------------------|--------------|-------------------|
| Full D3                                 | fallback→Ambivalent: | 0/300 (0.0%) | no-label-field: 4 |
| D3 without Gap Agent                    | fallback→Ambivalent: | 0/300 (0.0%) | no-label-field: 2 |
| D3 with Merged Q-A Agent                | fallback→Ambivalent: | 0/300 (0.0%) | no-label-field: 7 |
| Full D3 - Revised Question Intent Agent | fallback→Ambivalent: | 0/300 (0.0%) | no-label-field: 3 |
| Full D3 - Revised Answer Agent          | fallback→Ambivalent: | 0/300 (0.0%) | no-label-field: 2 |
| D3 Gap robust to Answer Agent           | fallback→Ambivalent: | 0/300 (0.0%) | no-label-field: 3 |

Figure 41: Invalid output and fallback inspection for the first set of experiments

The invalid-output inspection for the first set confirms the same picture as the second set: fallback predictions were not an issue. Every variant produced zero fallback-to-Ambivalent cases on the 300 examples. Only a small number of outputs per variant lacked a clean label field in their parsed JSON (at most 7 for the merged Question-Answer variant), and in all of these cases the label was still recovered from the raw text. Therefore, the performance differences in this set are driven by genuine wrong-label decisions and not by malformed output formatting.

**7.3.2. Second Set.** The table shows that the best-performing variant in terms of weighted F1-score is **D3 + KNN Few-Shot**, with a weighted F1-score of 0.50. This is slightly higher than the Full D3 baseline, which achieves 0.48 on the same 100-example subset. The KNN few-shot variant also improves the macro F1-score from 0.32 to 0.41, indicating better balance across the three classes.

Table 4: Comparison of D3 variants on the validation set (sorted by weighted F1).

| Model                              | Acc. | Prec. Macro | Rec. Macro | F1 Macro    | Prec. W. | Rec. W. | F1 W.       |
|------------------------------------|------|-------------|------------|-------------|----------|---------|-------------|
| D3 + KNN Few-Shot                  | 0.51 | 0.48        | 0.39       | <b>0.41</b> | 0.50     | 0.51    | <b>0.50</b> |
| Full D3                            | 0.52 | 0.31        | 0.34       | 0.32        | 0.45     | 0.52    | 0.48        |
| D3 + DSPy-Optimized Gap Agent      | 0.49 | 0.32        | 0.35       | 0.33        | 0.46     | 0.49    | 0.47        |
| D3 + DSPy-MIPROv2 Decision         | 0.42 | 0.40        | 0.36       | 0.34        | 0.46     | 0.42    | 0.41        |
| D3 + DSPy-Optimized Decision Agent | 0.28 | 0.35        | 0.49       | 0.27        | 0.43     | 0.28    | 0.21        |

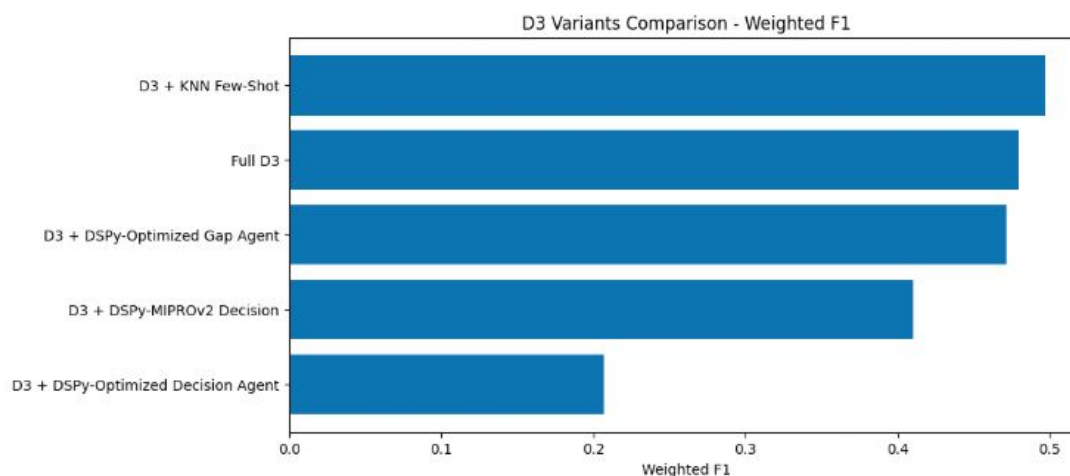


Figure 42: Weighted F1 comparison for the second set of D3 variants



The invalid-output inspection shows that fallback predictions were not a major issue in this set of experiments. All variants produced zero fallback-to- Ambivalent cases. Only the KNN few-shot variant and the DSPy-optimized Decision Agent produced one missing-label field each. Therefore, the differences in performance are mainly caused by wrong label decisions rather than by invalid output formatting.

|                                    |                      |              |                   |
|------------------------------------|----------------------|--------------|-------------------|
| Full D3                            | fallback→Ambivalent: | 0/100 (0.0%) | no-label-field: 0 |
| D3 + KNN Few-Shot                  | fallback→Ambivalent: | 0/100 (0.0%) | no-label-field: 1 |
| D3 + DSPy-Optimized Gap Agent      | fallback→Ambivalent: | 0/100 (0.0%) | no-label-field: 1 |
| D3 + DSPy-Optimized Decision Agent | fallback→Ambivalent: | 0/100 (0.0%) | no-label-field: 0 |
| D3 + DSPy-MIPROV2 Decision         | fallback→Ambivalent: | 0/100 (0.0%) | no-label-field: 0 |

Figure 43: Invalid output and fallback inspection for the second set of experiments

## 8. Error Analysis

**Format errors.** Formatting failures were negligible. On both validation subsets, every Decision Agent output was mapped to one of the three valid labels, so the pipeline never had to fall back to Ambivalent. A few outputs per variant were missing a clean label field in their JSON (at most 7 out of 300 for the merged Question–Answer variant on the first subset, and one each for the KNN and DSPy-Gap variants on the second subset), but in every case the label was recovered from the raw text. The Gap and Evasion Agent sometimes returned incomplete JSON, which was fixed automatically by the `_close_json` step (Section 6.1). Overall, the performance differences between variants come from wrong label decisions, not from malformed output.

**Class-separation errors.** In almost all variants the same error pattern appears on both subsets. The two decisive classes, Clear Reply and Clear Non-Reply, are most often confused with the majority Ambivalent class, while direct confusion between Clear Reply and Clear Non-Reply is rare. In other words, the pipeline tends to push uncertain answers into the middle category instead of committing to one of the two decisive labels.

This behaviour has two main causes. First, because of the class imbalance, Ambivalent is the safest default, so Clear Non-Reply, the minority class, is the hardest to separate and is almost always absorbed into Ambivalent; this is the reason for the consistently low Clear Non-Reply recall reported throughout Section 7. Second, the boundary between Clear Reply and Ambivalent depends on how cautious each variant is. Variants that keep the full reasoning chain are more cautious and tend to label borderline answers as Ambivalent, whereas removing the Gap and Evasion Agent or merging the first two agents makes the pipeline over-confident and shifts these answers toward Clear Reply (the corresponding counts were given in Sections 7.1.2 and 7.2.3).

The only clear exception is the Decision Agent optimised with BootstrapFewShot (Section 7.2.3), which behaves in the opposite way: instead of defaulting to Ambivalent, it almost stops predicting that class altogether, with its Ambivalent recall dropping to 0.03 and most of those answers reassigned to Clear Reply or Clear Non-Reply. This shows that the preference for the middle category is not inherent to the task but is driven by the cautious decision rule: once the optimisation pushes the model to recognise the minority class, the imbalance is simply reversed and the overall weighted-F1 collapses instead of improving.

**Error rate by length.** The error rate does not depend strongly on input length. For the full D3 baseline it stays close to the overall error rate across all four buckets, both for answer length (between 0.41 and 0.49) and for question length (between 0.40 and 0.49), so neither very short nor very long inputs are mainly responsible for the mistakes. The only mild exception is the bucket of medium-short answers (about 60–200 words), which has the highest error rate for the baseline (0.49) and is the hardest bucket for most of the variants.

This fits the exploratory analysis, where the classes already differ in length: Ambivalent answers are the longest (about 332 words), Clear Reply ones are in the middle (about 272 words), and Clear Non-Reply ones are the shortest (about 138 words). So length is mostly correlated with the class, and each length bucket simply contains a different mix of classes. Since the main error is class confusion — the decisive classes being pushed into the majority Ambivalent class — the error rate reflects how well the classes are separated, not the length itself.

**Qualitative examples.** The example errors below illustrate the three dominant patterns. In the first, the answer does give concrete reasons, but the cautious pipeline reads the discursive framing as only partially responsive and downgrades a Clear Reply to Ambivalent:

```
True: Clear Reply | Predicted: Ambivalent
Q: What gives the speaker confidence today to believe that the Prime Minister
 can
 achieve what he hasn't done over the last 6 months?
A: Well, as you mentioned, he's been in power for 6 months, and I've been able
 to
 watch a leader emerge. The first thing that gives me confidence is that he
 wants
 responsibility. A sign of leadership is for somebody to say, I want to be
 able to...
```

In the opposite case, a fluent, on-topic answer poses and answers its own sub-question with an explicit “Yes, I think he will”, without ever stating whether the speaker’s view has actually changed. The pipeline reads this affirmative as a direct reply, so a true Ambivalent is predicted as Clear Reply:

```
True: Ambivalent | Predicted: Clear Reply
Q: Has your view of Putin changed in the past few months, considering the
 recent
 developments in Ukraine?
A: The answer is that I think he still does not want any full-blown war,
 number one.
 Number two, do I think he'll test the west, test the United States and NATO
 as
 significantly as he can? Yes, I think he will. But I think he'll pay a
 serious...
```

Finally, a Clear Non-Reply is predicted as Ambivalent because, although the answer effectively claims ignorance, it still engages with the topic and adds related commentary:

```
True: Clear Non-Reply | Predicted: Ambivalent
```

Q: Does it require signing?  
A: You know, it raises an interesting question as to whether technically there's actually a signature, since, as I said, it's not a legally binding agreement. I don't know what the protocols are. But I do think that this is a commitment that...

These examples match the aggregate counts: the recurring failure mode is an answer that ends with a brief affirmative or restates the topic without actually resolving the target sub-question.

## 9. Best Model Selection

Across both experimental sets, the variants were compared primarily using weighted-F1, since this metric accounts for the imbalanced label distribution of the dataset. Macro-F1 was also monitored in order to evaluate how well each variant handled the minority class, Clear Non-Reply.

Among all tested configurations, the best-performing variant was the full D3 pipeline with the revised Answer Content Agent. This variant achieved a weighted-F1 score of 0.571 and a macro-F1 score of 0.491, obtaining the highest score on both metrics. Compared with the baseline full D3 pipeline, which achieved 0.533 weighted-F1 and 0.407 macro-F1, this improvement shows that the revised Answer Content Agent had a positive effect on the overall pipeline.

The main change in this variant was that the Answer Content Agent was made more conservative. In particular, the revised prompt applied a stricter definition of what counts as a direct answer and encouraged political rhetoric, background claims, and unrelated details to be placed in `off_target_content`. This improved the quality of the intermediate representation passed to the later stages of the pipeline. By distinguishing more clearly between directly relevant information and content that was only partially related or insufficient, the revised agent provided the Gap and Evasion Agent and the Decision Agent with more reliable evidence. As a result, the final classification decisions became less dependent on potentially inaccurate interpretations of the answer and more consistent with the actual relation between the target question and the response.

The D3 + kNN Few-Shot variant was also considered as a strong candidate, with a weighted-F1 score of 0.497 and a macro-F1 score of 0.413. However, this variant was not selected mainly because of runtime limitations. For each validation or test instance, it computes a sentence embedding, retrieves the nearest neighbours from the training pool, and constructs a dynamic prompt with instance-specific examples. This makes the method considerably more expensive to run over the full test set in the limited Kaggle environment. In addition, the kNN result was measured on a different and smaller validation split than the manually designed D3 variants, so the scores are not directly comparable. Therefore, although the kNN-based approach could plausibly reach comparable or even better performance, especially for borderline Clear Reply versus Ambivalent cases, it was not chosen as the final configuration.

The selected final configuration therefore keeps the Question Intent Agent, the Gap and Evasion Agent, and the Decision Agent with their original prompts, while replacing only the Answer Content Agent with its revised version. Since the validation set is

relatively small, the absolute differences between variants should be interpreted with some caution. Nevertheless, this configuration offered the best balance between predictive performance, robustness, and runtime cost, and was therefore selected as the final model for producing the held-out test-set predictions and the Kaggle submission.

## 10. Comparison with the Previous Projects

The experiments across all four projects show that the CLARITY task cannot be solved by detecting specific words or repeated phrases in the answer. The central challenge is to decide whether a response actually resolves the target sub-question in a complete and clear way: a politically detailed answer may still be only partially responsive, while a short answer may be sufficient if it directly addresses what was asked. The classification therefore depends on the relation between the target sub-question and the answer, rather than on fixed lexical cues. This relation is hard to model for both traditional classifiers and language-model-based approaches, and it is made harder by the class imbalance, since the minority class, Clear Non-Reply, is the most difficult to identify reliably.

In the first project, the task was approached with traditional vector-based representations, mainly TF-IDF and Word2Vec, combined with Logistic Regression. The best TF-IDF configuration achieved the strongest raw scores of that project, with a macro-F1 of 0.5332 and a weighted-F1 of 0.5965 under 5-fold stratified cross-validation, but its learning curves indicated overfitting, suggesting that the model partly relied on dataset-specific lexical patterns rather than on robust semantic generalization. The Word2Vec experiments produced slightly lower scores, with the lemmatized Word2Vec model reaching a macro-F1 of 0.5019 and a weighted-F1 of 0.5703; because it showed more stable training-validation behaviour, it was selected as the final configuration of that project despite TF-IDF's higher metrics. In both cases the representations remained fundamentally limited: TF-IDF captures lexical frequency, while Word2Vec provides static word-level embeddings, and neither models the relation between a specific sub-question and its answer. This matters for CLARITY, where the same answer can receive a different clarity label depending on which sub-question is evaluated. The first project therefore established that traditional representations are useful baselines but cannot capture the semantic question-answer relation.

In the second project, the task was approached with supervised fine-tuning of Transformer-based encoders, mainly BERT, DistilBERT, and DeBERTa. This setting produced the strongest overall results among all projects, because the models were trained directly on the labelled dataset and could learn task-specific patterns from the question-answer pairs. Unlike TF-IDF and Word2Vec, these models produce contextual representations in which the meaning of each token depends on its surrounding text, which is essential for judging whether the answer addresses the specific target sub-question.

Among the tested models, BERT offered the best balance between performance, stability, and cost. The baseline BERT model was already strong, with a mean macro-F1 of 0.6009 and a weighted-F1 of 0.65 under 4-fold cross-validation. After testing several variants (different batch sizes, learning rates, weighted losses, a striding-window input, more epochs, and learning-rate scheduling), the best configuration was Experiment 10, which used weighted cross-entropy loss with class weights [1.3, 1.0, 1.5] and a warm-up scheduler, reaching a mean macro-F1 of 0.6041 (std 0.0162), an accuracy of 0.65, and a weighted-F1 of 0.64. The gain over the baseline was small, but this con-

figuration was selected because it slightly improved macro-F1 while remaining stable across folds. DistilBERT was close behind, with a best mean macro-F1 of 0.5940 (std 0.0137), making it an efficient alternative when training time or resources are limited. DeBERTa was the hardest to fine-tune: its baseline produced NaN losses and a macro-F1 of only 0.1562, and even after stabilization (lower learning rate, float32 precision, warm-up, stronger class weights, more epochs, and gradient clipping) its best configuration reached a mean macro-F1 of 0.5028 (std 0.0335), below both BERT and DistilBERT. Overall, the second project showed that supervised fine-tuning was the most effective strategy, and that model complexity alone did not guarantee better performance, since the simpler BERT-based models were more reliable on this dataset.

The third project moved from fine-tuning to prompting. Instead of updating model parameters, pretrained instruction-tuned models were used directly through prompts, which made the approach lighter and more flexible but also more sensitive to prompt wording, label bias, and output-format inconsistencies. Several strategies were tested, including zero-shot, semantic kNN few-shot, and Chain-of-Thought prompting. In the zero-shot setting, the best result came from Qwen3.5-0.8B (accuracy 0.515, macro-F1 0.447, weighted-F1 0.520), but performance was not stable across model sizes: the larger Qwen3.5-2B and Qwen3.5-4B showed stronger label bias, especially toward Clear Reply.

The most effective strategy was semantic kNN few-shot prompting, where demonstrations were selected dynamically from the training pool by semantic similarity to the input. For the official weighted-F1 objective, the most practical configuration was Qwen3.5-0.8B with  $K=3$  unbalanced examples (accuracy 0.582, macro-F1 0.416, weighted-F1 0.538), which was selected as the final prompting configuration. Larger models benefited more from semantically selected examples on macro-F1: Qwen3.5-2B reached 0.470 and Qwen3.5-4B obtained the best macro-F1 of the project (0.479, with accuracy 0.496 and weighted-F1 0.490). Chain-of-Thought prompting did not help, reaching only 0.319 accuracy, 0.290 macro-F1, and 0.223 weighted-F1 for Qwen3.5-0.8B while increasing runtime. The third project therefore established prompting as a flexible but less reliable alternative to fine-tuning, motivating the D3-Agentic pipeline, which decomposes the decision into more structured reasoning steps.

The current D3-Agentic approach extends this prompting direction by decomposing the classification into question-intent analysis, answer-content extraction, gap/evasion analysis, and a final decision. This makes the reasoning more interpretable and allows errors to be traced to an intermediate stage. The experiments showed that the Answer Content Agent and the Gap and Evasion Agent most strongly affect the final prediction: if the Answer Content Agent treats a vague political statement as a direct answer, or the Gap and Evasion Agent misses that only part of the question is addressed, the Decision Agent receives misleading evidence and may assign an incorrect final label.

The best D3 variant, the full pipeline with the revised Answer Content Agent, reached a weighted-F1 of 0.571 and a macro-F1 of 0.491, higher on both metrics than the Qwen3.5-0.8B kNN few-shot result of the third project. This suggests that explicitly separating the analysis of the question, the answer, and the gap between them yields more controlled decisions than a single prompt. The D3 system nevertheless remains below the fine-tuned encoders of the second project, which is expected: D3 updates no parameters and relies only on the model's instruction-following ability, whereas fine-tuned BERT learns task-specific patterns directly from the labelled data.



Table 5 summarises the final (selected) configuration of each project.

Table 5: Cross-project comparison on the CLARITY task, showing the selected final configuration of each project. For the first project the lemmatized Word2Vec model is reported, which was chosen for its more stable training behaviour even though TF-IDF obtained slightly higher raw scores (macro-F1 0.53, weighted-F1 0.60). Evaluation protocols differ across projects, so the scores reflect a qualitative trade-off rather than a strict ranking.

| Project                | Approach (final config)        | Protocol    | Macro-F1 | Weighted-F1 |
|------------------------|--------------------------------|-------------|----------|-------------|
| 1. Traditional ML      | Word2Vec (lemmatized) + LogReg | 5-fold CV   | 0.50     | 0.57        |
| 2. Fine-tuned encoders | BERT (w-CE + warm-up)          | 4-fold CV   | 0.60     | 0.64        |
| 3. LLM prompting       | Qwen3.5-0.8B, kNN FS ( $K=3$ ) | val. subset | 0.42     | 0.54        |
| 4. D3-Agentic          | Full D3 + Revised Answer Agent | val. (300)  | 0.49     | 0.57        |

Overall, the four projects reveal a clear trade-off between performance, interpretability, and computational cost. Traditional TF-IDF and Word2Vec models are simple and fast but limited to lexical patterns, so they cannot reliably separate answers whose label depends on the target sub-question. Fine-tuned Transformers achieved the strongest performance, capturing the semantic question–answer relation better than any other approach, but at the cost of training, validation, class-weight decisions, and hyperparameter tuning. Standard prompting required no training and adapted easily to new formulations, but was less stable and prone to over-predicting the majority class or confusing partially responsive answers with clear non-replies.

The D3-Agentic pipeline improved over the simple prompting baseline, but the gain was modest rather than decisive. Its main value is not raw performance but a more structured and diagnosable decision process: an error can be traced to a misunderstood question, an incomplete or misleading answer interpretation, an underestimated gap, or a wrong final decision. This interpretability comes at a cost, since each prediction requires several sequential LLM calls, which slows inference and experimentation. D3 is therefore best seen as a middle ground between simple prompting and supervised fine-tuning, more interpretable and slightly more effective than the former, but less efficient and still below the latter.

Finally, the numerical comparison should be interpreted with caution, because the experiments were not evaluated under identical protocols: the traditional and fine-tuned models used 5-fold and 4-fold stratified cross-validation over the full training set, whereas the prompting and D3 experiments used smaller held-out validation subsets. The comparison is therefore most useful for understanding the strengths and limitations of each strategy rather than for producing a strict ranking based only on the reported scores.

Taken together, these results highlight how genuinely difficult the CLARITY task is. Across four very different modelling strategies, ranging from simple lexical baselines to supervised fine-tuning and multi-agent prompting, performance remained within a relatively narrow band, and even the strongest approach, fine-tuned BERT, only reached a macro-F1 of about 0.60. No method came close to solving the task, and every one of them struggled most with the minority Clear Non-Reply class. This consistency across approaches indicates that the main obstacle is not the choice of model but the nature of the problem itself: deciding whether an answer truly resolves a specific target

sub-question requires a relational, pragmatic judgement that is hard to learn from lexical features, hard to fine-tune under class imbalance, and hard to elicit reliably through prompting alone.